

中国研究生创新实践系列大赛

“华为杯”第二十二届中国研究生

数学建模竞赛

题 目： 无监督迁移学习下的轴承故障诊断建模与优化

摘 要：

轴承作为高速列车走行系统中的关键旋转部件，在复杂工况下容易损坏，因此发掘轴承故障信号特征以及进行精准智能故障诊断具有重要意义。迁移学习能够利用台架实验数据作为源域知识迁移到实际运营环境，有效缓解样本不平衡并提升故障诊断的工程适用性。

针对问题一，首先进行数据筛选，从数据集中选取采样率为 48 kHz 的驱动端振动信号，并重采样至 32 kHz。其次，采用两阶段的 TVFEMD 与 GWO-MOMEDA 组合进行降噪处理，随后，进行 SMOTE 实现样本平衡。最后，对信号进行切片并进行特征提取，分别从时域、频域与熵域构建 36 项特征指标，并进一步引入 MixHop 实现多域特征融合，实现对轴承状态信息的综合表征。

针对问题二，首先在第一问的多域融合特征数据集上进行归一化与标准化。接着对数据集进行 8:2 的随机划分，前者用于算法训练，后者用于验证算法准确率。本文全面评估了随机森林、长短期记忆网络、卷积神经网络以及 Vision Transformer 算法在故障诊断上的表现，并通过性能和稳定性分析选择了最优的 Vision Transformer 模型。作为本问及后续的基础模型，准确率达到 95.7%。

针对问题三，首先利用核密度估计对源域与目标域的特征分布进行对比，确认两者存在显著分布偏移。随后，基于最大均值差异与统计特征变换的度量，提出跨数据空间、跨阶数的二阶与高阶统计量联合匹配策略。在此基础上，构建基于 Vision Transformer 的无监督领域自适应网络，通过二阶与高阶矩的联合对齐缩小源域与目标域的分布差距，并借助权值共享机制促进可迁移特征的学习，从而实现高效的领域自适应。

针对问题四，首先通过特征空间的可视化与定量度量，呈现迁移前后由“散乱到对齐”的跨域特征演化，直观展示对齐效果；其次，从质心一方差对齐与距离分布出发，量化域级统计对齐程度与残余偏差，并评估其对跨域鲁棒性的贡献；最后，结合注意力可视化开展事后可解释性分析，对单次预测的证据与可信度进行审计，并与轴承故障机理进行一致性验证。

关键词： 轴承故障诊断、TVFEMD 算法、MixHop、SMOTE、高阶与二阶联合矩匹配

目录

1 问题重述	4
1.1 问题背景	4
1.2 数据介绍与建模目标	4
1.2.1 数据介绍	4
1.2.2 建模目标	5
1.3 问题提出	5
1.4 解题流程	6
2 模型假设	7
3 符号说明	7
4 问题 1: 基于自适应 MOMEDA 与 MixHop 的多域融合特征矩阵构建	7
4.1 问题分析	7
4.2 数据筛选	8
4.3 数据预处理	9
4.4 数据去噪	10
4.5 特征提取	18
4.5.1 时域特征	19
4.5.2 频域特征	20
4.5.3 熵域特征	21
4.5.4 基于 MixHop 模型构建特征矩阵	21
5 问题 2: 基于 Vision Transformer 的轴承故障诊断模型构建	24
5.1 问题分析	24
5.2 模型建立	24
5.2.1 Vision Transformer (ViT)	24
5.2.2 卷积神经网络 (Convolutional Neural Network, CNN):	25
5.2.3 长短期记忆网络(Long Short-Term Memory, LSTM):	26
5.2.4 随机森林(Random Forest, RF):	26
5.3 模型求解	27
5.3.1 模型参数设置	27
5.3.2 数据集准备	28
5.3.3 评价指标	28
5.3.4 模型性能分析	29
5.3.5 模型稳定性分析	31
6 问题 3: 无监督领域自适应网络迁移诊断	31
6.1 问题分析	31
6.2 融合高阶与二阶矩匹配的无监督深度领域适应网络	32
6.2.1 分布差异度量	32
6.2.2 高阶与二阶联合矩匹配方法	33
6.2.3 基于 Vision Transformer 的无监督领域自适应网络设计	34
6.3 模型训练	35
6.4 结果可视化分析	37
7 问题 4: 基于迁移学习的轴承故障诊断可解释性分析	38
7.1 问题分析	38
7.2 迁移过程可解释性分析	39

7.2.1 从散乱到对齐的跨域特征演化	39
7.2.2 质心一方差对齐与距离分布	40
7.3 事后可解释性分析	41
8 模型的优缺点分析	42
8.1 模型的优点	42
8.2 模型的缺点	42
9 参考文献	43
10 附录	44

1 问题重述

1.1 问题背景

随着我国高速铁路技术的快速发展，高速列车正以其安全高效、便捷舒适、绿色低碳等特点，逐渐成为绝大多数市民出行的首要选择。日均数以百万计的乘客信赖并选择高铁出行，这不仅体现了其作为现代交通工具的巨大成功，还对列车的可靠性和安全性提出了很高的要求。然而，在这光鲜的背后，高速列车走行系统中的关键旋转部件——轴承，却构成了运营安全与效率的一大潜在隐患。

轴承主要结构为内圈、外圈、滚动体和保持架四部分，其中典型故障多发生在内圈、外圈和滚动体这三个核心承载部件。轴承作为高速列车走行系统的关键旋转部件，其工作环境复杂恶劣，长期承受着高转速、交变载荷以及复杂振动冲击等多重挑战。这种严苛的工况，使得轴承部件可能因过热、润滑不良、腐蚀等多种因素发生损坏，进而影响列车正常运行。一旦轴承出现故障，其后果将是灾难性的，轻则导致列车在高速运行中发生异响、振动加剧，影响旅客出行体验及铁路运营效率；重则可能诱发更严重的恶性事故，如车轴断裂、轮对脱轨等，直接危及乘客的生命安全，造成巨大的经济损失和社会影响。因此，对高速列车轴承进行实时、精准的状态监测与故障诊断，是确保列车安全、高效运营的重中之重^[1]。

目前高铁轴承状态的监测方法主要依赖于传统的信号处理技术，结合专家经验构建的特征指标进行故障诊断。这些方法通常包括振动信号的时域、频域分析，通过提取均方根值、峰值、峭度、谐波分量等特征来判断轴承的健康状况。在早期，这些方法在一定程度上发挥了作用，为铁路运输的安全提供了基础保障。然而，随着中国轨道交通系统进入快速发展时期，列车运行密度持续增加，运营场景日趋复杂，智能化、强耦合度的发展趋势对轴承故障诊断技术提出了前所未有的高要求。现有传统方法在诊断精度、泛化能力和实时性方面，难以满足复杂运营场景下对轴承故障进行精准、高效诊断的需求^[2]。

近年来，随着深度学习技术的飞速发展，数据驱动的智能故障诊断方法在工业装备运维领域，尤其是在列车装备运维领域受到了广泛关注。深度学习模型，以其强大的特征学习能力和模式识别能力，能够从海量的运营数据中自动提取高维、抽象的故障特征，实现远超传统方法的故障识别精度。

然而，将深度学习模型应用于高速列车轴承的实际故障诊断，面临着两项核心挑战：复杂运行环境对传感器原始信号的影响；真实故障数据的极度稀缺与数据分布严重不平衡。

在此背景下，迁移学习技术为解决上述两大难题提供了解决方案。迁移学习作为一种新兴的机器学习方法，其核心思想是将在一个任务或领域中学到的知识，迁移并应用于另一个相关但不同的任务或领域，以提升目标任务的模型性能。对于高速列车轴承故障诊断而言，这意味着可以充分利用数据量丰富、标签完善的台架实验数据（源域）来训练一个初步的故障诊断模型，然后将该模型学习到的通用故障知识迁移到数据稀缺、分布不平衡的实际运营列车数据（目标域）上，通过少量的目标域数据进行微调，从而有效缓解真实故障数据稀缺和样本不平衡的问题，最终实现更高精度、更强泛化能力和更高效的实时诊断。

1.2 数据介绍与建模目标

1.2.1 数据介绍

本文给出 161 个具有代表性的轴承试验台架振动数据文件作为问题研究的源域数据集，另外给出来自实际的 16 个轴承故障文件作为目标域数据集

表 1-1 数据文件介绍表

序号	文件名	主要内容
1	目标域数据集	该数据集包含列车滚动轴承外圈（OR）、内圈（IR）、滚动体（B）故障和正常状态（N）下的振动信号数据，采集时间为 8 秒，采样频率为 32kHz，列车速度约 90km/h（轴承转速约 600 rpm）。数据文件以英文字母 A~P 编号命名，各数据所属工作状态未知。
2	源域数据集	该数据集是在受控的实验室台架实验环境下采集到的轴承数据，包含驱动端和风扇端轴承在 12KHz 和 48KHz 的采样频率下，4 种工作状态不同故障尺寸所采集的振动信号数据

1.2.2 建模目标

利用迁移学习方法，实现从试验台架数据到实际列车数据的跨域轴承故障智能诊断，并确保诊断模型具备较高的准确性、鲁棒性和可解释性。

1.3 问题提出

基于上述附件数据与机械相关的知识背景，本文需解决下列四个问题：

问题 1：数据分析与故障特征提取

从台架试验源域数据中筛选具有代表性的数据样本，结合轴承故障机理选择合适的特征提取方法，对源域数据进行特征分析以及特征提取，为后续诊断任务提供输入特征。

问题 2：源域故障诊断

在问题 1 中提取的故障特征基础上，对源域数据集进行划分，划分为训练集与测试集。对不同故障类型进行分类识别，并对诊断精度和性能进行验证与评价。

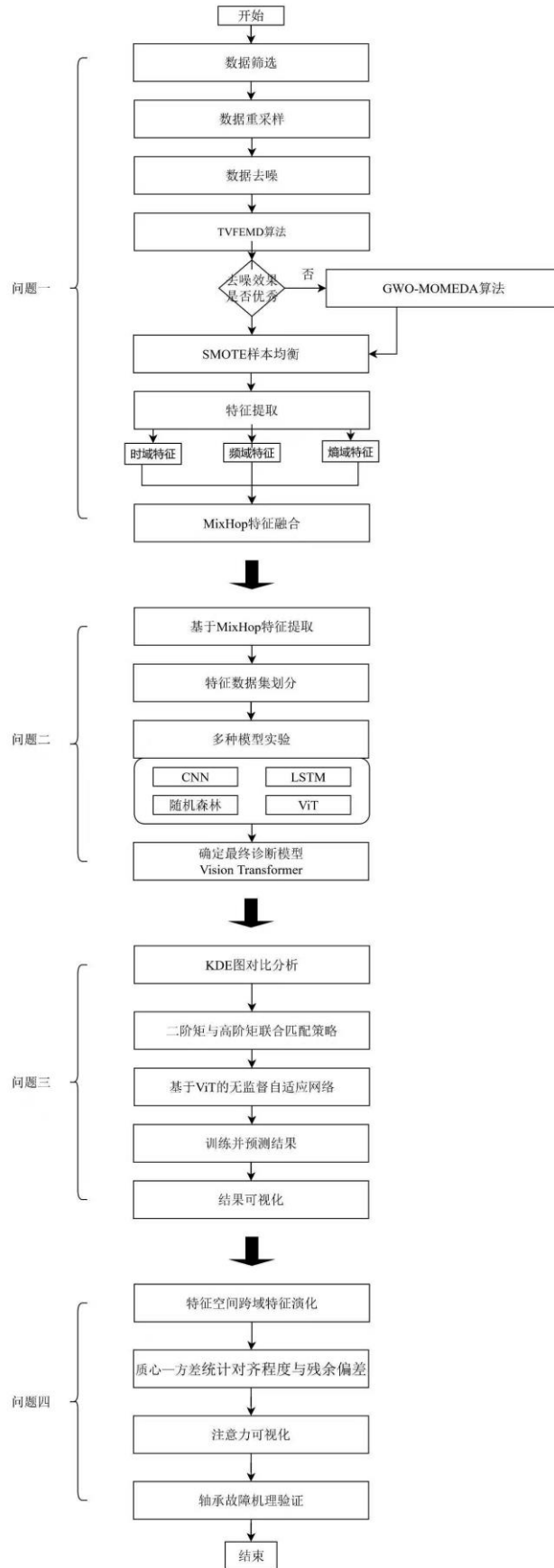
问题 3：目标域迁移诊断

在问题 2 对模型故障诊断基础上，针对源域与实际列车目标域数据在分布和数量上的差异，设计合适的迁移学习策略，将源域知识迁移到目标域，实现对目标域轴承故障的分类与标注，并进行结果可视化与分析。

问题 4：迁移诊断的可解释性

结合问题 3 中模型的结构设计、迁移过程和决策过程，以及轴承故障特点与机理，对迁移诊断的事前、迁移过程或事后（任选其一或多个方面）可解释性进行分析。提升迁移诊断的透明性与可验证性，避免模型在跨工况/跨设备场景中沦为“黑箱”。

1.4 解题流程



2 模型假设

假设一：在同一时间点轴承只存在一个故障。

假设二：假设实验台架数据与实际高速列车运行数据在故障机理层面具有相似性。

假设三：除赛题所给因素外，其他外界因素如传感器老化在误差允许范围内可忽略。

3 符号说明

序号	符号	含义
1	f_o	外圈故障特征频率
2	ω	特征提取窗口
3	X	原始振动信号
4	d	特征维度
5	n	轴承内圈转速
6	$U = (u_1, u_2, \dots, u_n)$	最优 IMF 组合信号

4 问题 1：基于自适应 MOMEDA 与 MixHop 的多域融合特征矩阵构建

4.1 问题分析

根据题目要求，考虑后续目标域的迁移任务，需结合轴承故障特征，对源域数据集进行数据预处理，选取合适的方法对整体数据集进行特征提取，以及对有代表性的源域数据进行特征分析。本节对该问题的解答可分为以下步骤：

（1）数据筛选阶段。源域数据集中包含驱动端和风扇端轴承在 12KHz 和 48KHz 的采样频率下，4 种工作状态下不同故障尺寸所采集的振动信号数据，我们从中筛选出 48kHz 频率下的驱动端信号。

（2）数据预处理阶段。首先，进行数据重采样，源域数据集中 DE 的加速度信号从 48kHz 变换为与目标域数据集一致的 32kHz；再使用 TVFEMD 算法，将原始轴承故障振动信号分解成多个 IMF 分量，选取包含故障信号较多的分量进行累加，完成第一阶段的去噪操作；再根据第一阶段得到的信号，结合自适应 MOMEDA 算法和灰狼优化算法（Grey Wolf Optimizer, GWO）进行第二阶段的去噪。

（3）特征提取阶段。主要从时域、频域和熵域三个维度进行信号特征的选择，共选取了 16 个时域特征指标，13 个频域特征指标和 7 个熵域特征指标。将步骤（1）中得到的最优 IMF 组合信号等分成多个小样本，并在每个小样本中提取上述的 36 个特征指标，将其拼接成一个特征矩阵。再基于 MixHop 模型对提取的特征矩阵进行融合优化。

具体流程如图 4.1 所示。

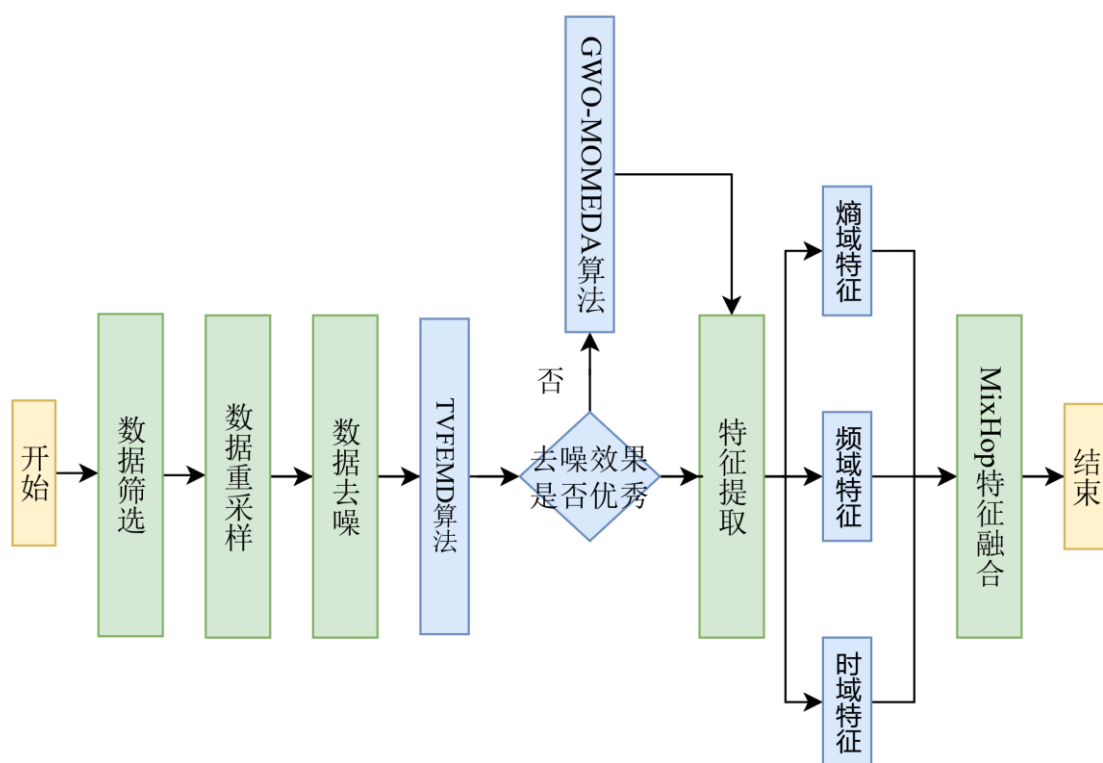


图 4.1 问题 1 思路流程图

4.2 数据筛选

为保证源域与目标域之间的特征可比性并最大化保留故障信息，本研究在源域数据中优先选择 48kHz 采样频率且来自驱动端（DE）的加速度信号。选择依据包括采样频率、传感器位置、信噪比与故障特征频带四个方面：

（1）采样频率与频谱信息完整性

轴承故障产生的冲击会激发高频共振模态，包络解调与谱峭度等诊断指标往往依赖于高频率信号特征。48kHz 的采样频率能覆盖比 12kHz 更宽的高频带，显著降低高频冲击与共振成分被混叠或频谱截断的风险。因此，48kHz 的数据在时域脉冲细节与包络谱分析上具有更高的信息保真度，有利于提取与故障相关的瞬时冲击特征与高阶谐波成分。

（2）测点一致性与传递路径

驱动端近轴承位置、传递路径短、振动幅值衰减小，能够更直接地反映轴承局部缺陷激发的冲击信号。源域中 DE 的数据在传递函数与测点耦合特性上与目标域高度一致，有助于降低测点引起的域间分布差异，从而减小源域到目标域迁移时的协变量偏移。

（3）故障可识别性

由于传感器位置靠近故障源，DE 信号通常具备更高的信噪比（SNR），冲击包络与瞬态能量更明显，利于基于包络谱、峭度等统计量的弱故障检测。相比之下，FE（风扇端）或基座（BA）信号经多层结构传递后高频成分显著衰减，且易混入结构或动力系统其它激励（如风扇叶片频率），导致特征冗余与域差异增大，不利于迁移学习。

（4）可处理性与后处理策略

选择 48kHz 源数据为上游准备提供了更大的后处理灵活性：可以在保持高频信息的前提下，通过适当的带限滤波并使用高质量的重采样将源数据下采样到目标域采样率（32kHz），以实现采样率一致化而不丢失关键故障带信息。相比之下，直接使用 12kHz 数据将迫使丢失高频诊断信息或进行风险更高的频谱外推，且难以通过插值恢复丢失信

息。

4.3 数据预处理

本研究需将源域 DE 加速度信号从 48kHz 变换为与目标域一致的 32kHz，以消除采样率差异导致的域偏差（sampling-induced domain shift），并使用 SMOTE 对数据集进行样本平衡，以改善分类器对少数类的识别能力。变换与采样的总体流程如下：

（1）高质量重采样：采用多相或基于窗函数/带限插值的重采样方法将 48kHz 转换到 32kHz（降采样比为 3:2），避免简单整除/插值带来的频谱畸变。

（2）幅值归一化与传感器响应校正：对源/目标信号做统一的幅值归一化，必要时进行传感器灵敏度或频率响应校正以减少测量链差异。

（3）特征验证：重采样后在源域上计算典型故障特征（外圈（OR）、内圈（IR）、滚动体（B）故障和正常状态（N）），并与原始 48kHz 结果对比，确认关键故障频率和包络能量保留良好；若发现显著损失，调整滤波器参数或考虑保留更高原始采样用于特征提取再进行特征级对齐。

（4）样本平衡：采用 SMOTE 算法，将数据集的每个类别的样本统一平衡到 28 个。

（5）窗口划分：查阅相关文献，选择窗口大小为 2048，步长为 1024，对样本进行窗口划分。

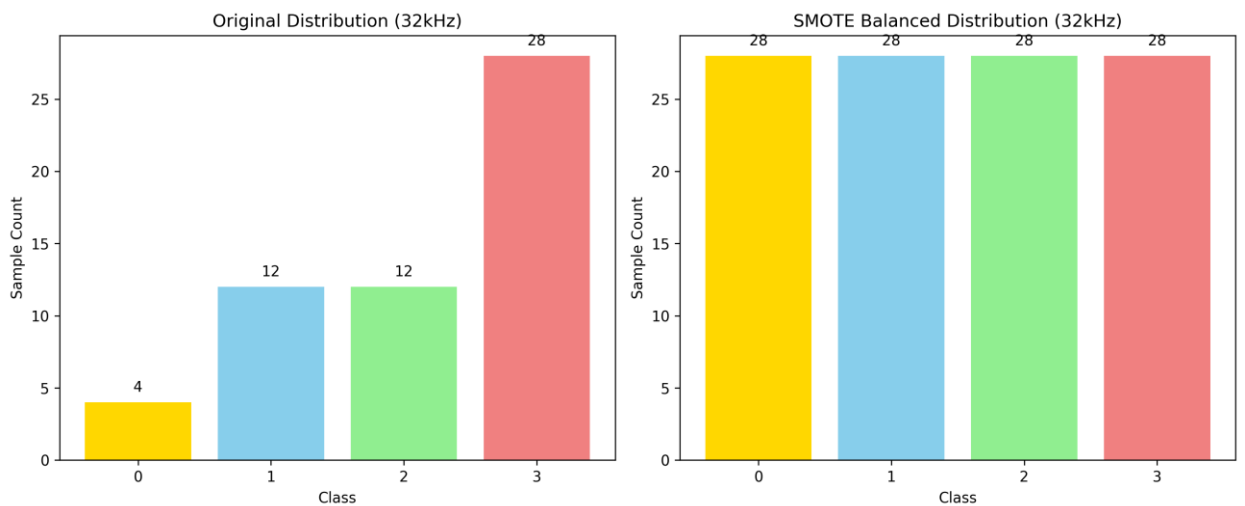


图 4.2 重采样特征对比图

图 4.2 中类别标签含义如表 4-1 所示。

表 4-1 故障类别与标签

类别标签	故障类别
0	正常状态
1	滚动体故障
2	内圈故障
3	外圈故障

4.4 数据去噪

在轴承运行过程中，其典型故障多发生在内圈、外圈和滚动体这三个核心部件。当轴承出现局部缺陷时，滚动体在与缺陷接触的瞬间会产生突变的冲击脉冲，因此在轴承周期性运转过程中，脉冲力会持续作用，从而会产生周期性的冲击分量。

题中源域数据集提供了试验平台中，驱动端轴承（SKF6205）在 12KHz 和 48KHz 以及风扇端轴承（SKF6203）在 12KHz 采样频率下的振动信号。其中轴承尺寸参数如表 4-2 所示。

表 4-2 源域数据集轴承尺寸参数

轴承类型	滚动体数 n	滚动体直径 d	轴承节径 D
SKF6205（DE）	9	0.3126 英寸	1.537 英寸
SKF6203（FE）	9	0.2656 英寸	1.122 英寸

上述轴承故障的频率特性通常会用故障特征频率来描述，该频率可以通过轴承转速、滚动体数量、滚动体直径等一系列参数计算得到，各故障的冲击频率计算公式如表 4-3 所示。

表 4-3 轴承故障特征频率计算公式

名称	特征频率
外圈故障特征频率（Ball Pass Frequency on the Outer Race, BPFO ）	$f_r \cdot \frac{N_d}{2} \cdot \left(1 - \frac{d}{D}\right)$
内圈故障特征频率（Ball Pass Frequency on the Inner Race, BPFI ）	$f_r \cdot \frac{N_d}{2} \cdot \left(1 + \frac{d}{D}\right)$
滚动体故障特征频率（Ball Spin Frequency, BSF ）	$f_r \cdot \frac{D}{d} \left[1 - \left(\frac{d}{D}\right)^2\right]$

表中， $f_r = \frac{n}{60}$ 为轴承转频， n 为轴承内圈转速（单位：rpm）， d 为滚动体直径， D 为轴承节径， N_d 为滚动体个数。各故障特征频率结果如表 4-4 所示。

表 4-4 轴承故障特征频率

外圈故障特征频率/Hz	内圈故障特征频率/Hz	滚动体故障特征频率/Hz
107	162	141

由于轴承部件结构功能与受力特点存在差异，所以在不同部位发生故障时，振动信号会呈现出不同的特征，如图 4.3 所示。

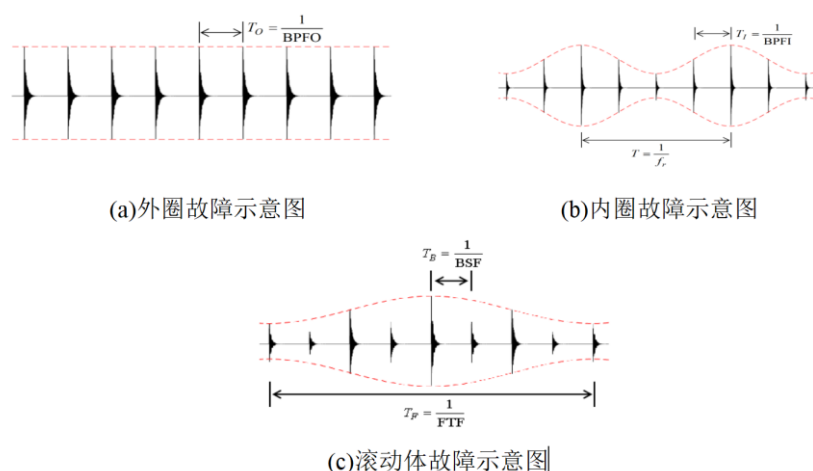


图 4.3 故障信号示意图

以外圈故障为例：

从时域上分析，如图 4.3(a)中所示预期轴承外圈故障信号示意图， T_o 为故障周期（相邻冲击的时间间隔），BPFO 为外圈故障特征频率，二者互为倒数。由于外圈通常固定不动，在载荷方向不变的情况下，外圈缺陷每次碰撞到滚动体的强度近似相同，所以理想情况下会在时域上呈现等周期等幅值的冲击分量。

下列图片如图 4.4，直观地呈现出通过加速度传感器采集到的有关轴承外圈故障（OR），内圈故障（IR）和滚动体故障（B）的故障信号以及其经过包络解调处理后的频谱图。然而在图 4.4(a)所示的轴承外圈故障信号图中可以清晰地看到，尽管存在故障，但在原始时域信号中并未展现出预期轴承外圈故障信号所预期的周期性冲击特征。并且整个波形杂乱无章，被大量的噪声和非周期性振动所淹没，导致故障引起的微弱冲击特征被掩盖，这种混杂的信号使得仅凭时域波形难以直接辨识出故障的存在与性质。

再从图 4.4(b)所示的轴承外圈故障包络谱来看。包络解调是传统上用于分析冲击性故障信号的有效手段，它旨在突出高频载波中的低频调制信息，从而揭示故障特征频率。然而，在(b)图中，包络谱中仅能勉强观察到一些较不明显的 3 倍和 7 倍 f_o （通常指外圈故障特征频率 BPFO 的倍频）。更为突出的是，包络谱中充斥着大量的宽带噪声和与故障本身无关的干扰频率分量，它们的幅值甚至与潜在的故障特征频率相近或更高。这无疑极大地干扰了故障特征的识别，使得诊断工作充满不确定性。图 4.5 和图 4.6 的情况也与外圈故障类似。

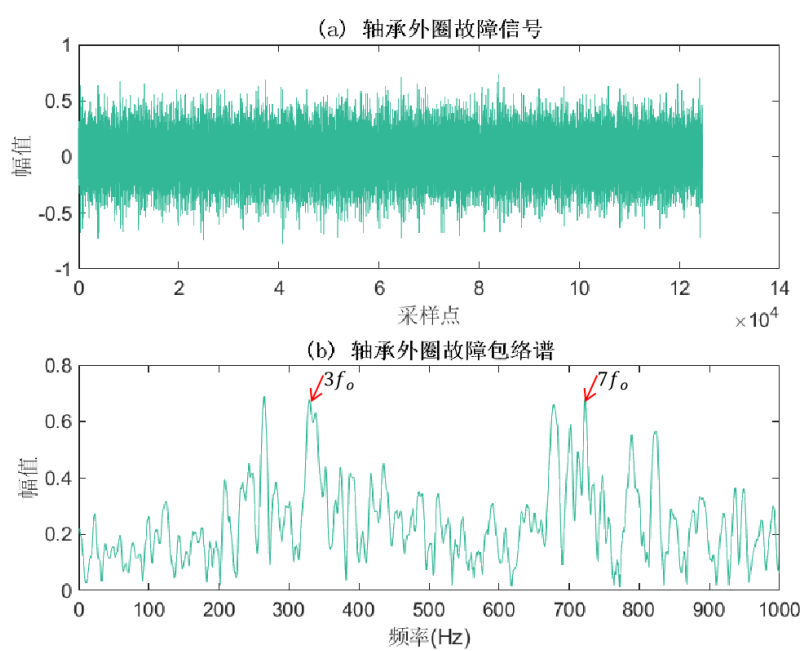


图 4.4 轴承外圈故障信号及其包络谱

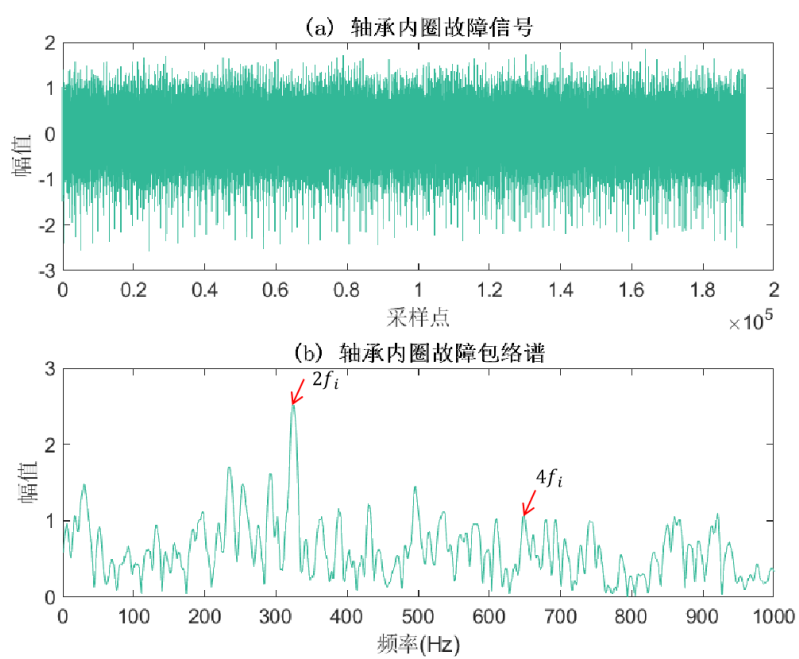


图 4.5 轴承内圈故障信号及其包络谱

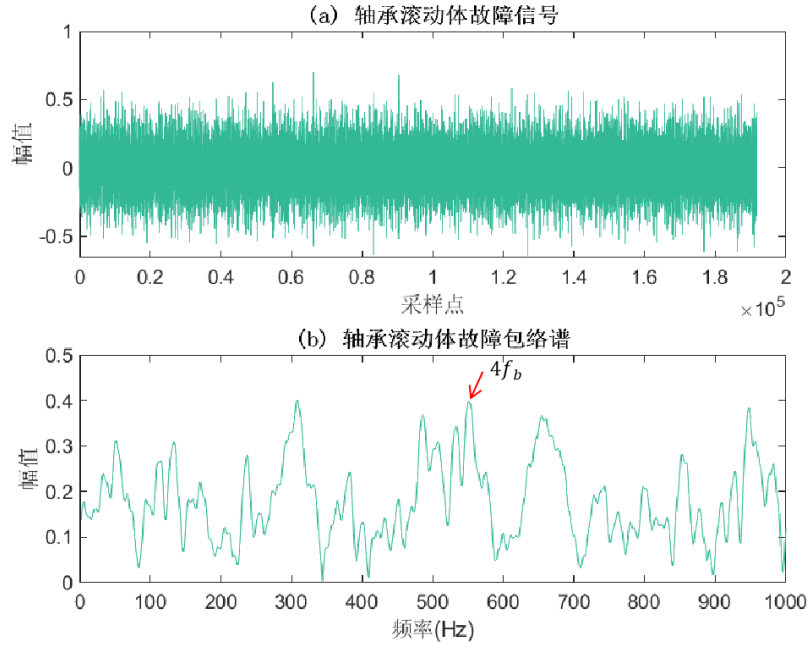


图 4.6 轴承滚动体故障信号及其包络谱

综合来看，这组图例说明了一个关键问题：在实际应用场景中，尤其是在高噪声、复杂工况下，传统的时域波形分析方法因其对噪声的敏感性而失效；而基于傅里叶变换和包络解调的包络谱方法，也因信噪比过低、干扰成分突出，难以有效地从原始故障信号中精确提取出关于滚动轴承健康状态的关键信息，这大大限制了故障诊断的准确性。

因此，对滚动轴承在复杂工况，多干扰环境下对微弱信号的预处理方法展开研究是十分必要的。传统的经验模态分解(Empirical Mode Decomposition, EMD)是一种自适应的信号处理方法，旨在将复杂的、非平稳、非线性的信号分解为有限个固有模态函数和一个残余分量^[3]。但是传统 EMD 还存在一些显著的局限性，比如：模态混叠、端点效应以及噪声敏感等。所以，本文针对上述问题，采用了时变经验模态分解(Time-Varying Filter Empirical Mode Decomposition, TVFEMD)^[4]技术对我们现有的源域数据进行数据去噪处理。

(1) TVFEMD 算法原理：

TVFEMD 的核心思想是构造一系列低通滤波器，对信号进行多次低通滤波，从而实现信号的自适应分解，但在构造滤波器时，其截止频率不是一成不变的，而是随着信号频率的改变而变化。设 $x(t)$ 为任意给定的双分量信号^[5]

$$x(t) = A(t)e^{j\varphi(t)} = a_1(t)e^{j\varphi_1(t)} + a_2(t)e^{j\varphi_2(t)} \quad (1)$$

采用 TVFEMD 对双分量信号 $x(t)$ 进行处理的步骤如下所示：

步骤 1: 对 $x(t)$ 进行 Hilbert 变换，根据变换结果计算信号的相位 $\varphi(t)$ 和幅值 $A(t)$ ：

$$z(t) = x(t) + j\hat{x}(t) = A(t)e^{j\varphi(t)} \quad (2)$$

步骤 2: 找到幅值 $A(t)$ 取到极小值和极大值分别对应的时刻 $\{t_{\min}\}$ 和 $\{t_{\max}\}$ 以及所对应

的幅值 $A(\{t_{\min}\})$ 、 $A(\{t_{\max}\})$ 。

步骤 3: 对幅值极小值点 $A(\{t_{\min}\})$ 进行插值以获得 $\beta_1(t)$, 对其极大值点 $A(\{t_{\max}\})$ 进行插值以获得 $\beta_2(t)$ 。

步骤 4: 计算瞬时均值 $\alpha_1(t)$ 和瞬时包络 $\alpha_2(t)$:

$$\alpha_1(t) = \frac{[\beta_2(t) + \beta_1(t)]}{2} \quad (3)$$

$$\alpha_2(t) = \frac{[\beta_2(t) - \beta_1(t)]}{2} \quad (4)$$

步骤 5: 通过插值 $\varphi'(t_{\min})A^2(t_{\min})$, $\varphi'(t_{\max})A^2(t_{\max})$ 得到 $\eta_1(t)$ 和 $\eta_2(t)$, 从而计算 $\varphi'_1(t)$ 和 $\varphi'_2(t)$,

$$\begin{cases} \eta_1(t) = \varphi'_1(t) [\alpha_1^2(t) - \alpha_1(t)\alpha_2(t)] + \varphi'_2(t) [\alpha_2^2(t) - \alpha_1(t)\alpha_2(t)] \\ \eta_2(t) = \varphi'_1(t) [\alpha_1^2(t) + \alpha_1(t)\alpha_2(t)] + \varphi'_2(t) [\alpha_2^2(t) + \alpha_1(t)\alpha_2(t)] \end{cases} \quad (5)$$

$$\begin{cases} \varphi'_1(t) = \frac{\eta_1(t)}{2\alpha_1^2(t) - 2\alpha_1(t)\alpha_2(t)} + \frac{\eta_2(t)}{2\alpha_1^2(t) + 2\alpha_1(t)\alpha_2(t)} \\ \varphi'_2(t) = \frac{\eta_1(t)}{2\alpha_2^2(t) - 2\alpha_1(t)\alpha_2(t)} + \frac{\eta_2(t)}{2\alpha_2^2(t) + 2\alpha_1(t)\alpha_2(t)} \end{cases} \quad (6)$$

再计算局部截止频率

$$\varphi'_{bis}(t) = \frac{\varphi'_1(t) + \varphi'_2(t)}{2} \quad (7)$$

步骤 6: 用局部截止频率 $\varphi'_{bis}(t)$ 对信号进行重构, 得到一个新的信号

$$h(t) = \cos \left[\int \varphi'_{bis}(t) dt \right] \quad (8)$$

步骤 7: 以 $h(t)$ 的极点为节点, 并将 $h(t)$ 划分为 n 段。再利用式(9)~(11)对信号进行 B 样条插值逼近, 将逼近结果 $m(t)$ 作为局部均值函数。

$$g_m^n(t) = [p_m^n \cdot x]_{\downarrow m} \cdot b_m^n(t) \quad (9)$$

$$p_m^n = [(b_m^m b_m^n)_{\downarrow m}]_{\uparrow m} \cdot b_m^n \quad (10)$$

$$b_m^n(t) = \beta^n \left(\frac{t}{m} \right), b_n^m(t) = \beta^m \left(\frac{t}{m} \right) \quad (11)$$

式中, $[\cdot]_{\downarrow m}$ 表示降采样, $[\cdot]_{\uparrow m}$ 表示过采样。

步骤 8: 根据式(12)~(14)计算 $\theta(t)$ 。对于给定的带宽阈值 ξ , 如果 $\theta(t) < \xi$, $x(t)$ 为 IMF;

否则, 令 $x(t) = x(t) - m(t)$, 重复步骤 1~步骤 7。

$$\theta(t) = \frac{B_{Loughlin}}{\varphi_{avg}(t)} \quad (12)$$

其中,

$$\varphi_{avg}(t) = \frac{a_1^2(t)\varphi_1'(t) + a_2^2(t)\varphi_2'(t)}{a_1^2(t) + a_2^2(t)} \quad (13)$$

$$B_{Loughlin} = \sqrt{\frac{(a_1'(t))^2 + (a_2'(t))^2}{a_1^2(t) + a_2^2(t)} + \frac{a_1^2(t)a_2^2(t)(\varphi_1'(t) - \varphi_2'(t))^2}{(a_1^2(t) + a_2^2(t))^2}} \quad (14)$$

(2) 模态分量的选择

通过 TVFEMD 对滚动轴承复合故障振动信号进行分解后, 可得到多个 IMF 分量。然而受噪声干扰及迭代误差累积的影响, 并非所有分量都具有实际意义, 通常仅有部分 IMF 与原始信号相关性较高, 包含了较好的故障冲击特征信息, 而其余分量多为噪声或虚假成分。一般而言, IMF 中所含故障冲击成分越多, 其包络谱分布偏离正态性的程度越大, 对应的峭度值也越高^[6]。因此, 包络谱峭度可作为衡量 IMF 中故障冲击成分的重要指标。但在强噪声背景下, 信号包络谱的峭度值可能对噪声引起的振幅离群值过于敏感, 可能导致其难以准确反映 IMF 的真实特征。

鉴于此, 本文构建加权包络谱峭度(Weighted Envelope Spectrum Kurtosis, WESK)指标^[4], 用于甄别含有丰富故障信息的有效分量和无效的噪声分量。加权包络谱峭度数学计算式为

$$ESK = \frac{E[(env_y - u_{env_y})^4]}{(E[(env_y - u_{env_y})^2])^2} \quad (15)$$

$$\rho(x, y) = \frac{E[(x - u_x)(y - u_y)]}{\sigma_x \sigma_y} \quad (16)$$

$$WESK = \rho(x, y) \cdot ESK \quad (17)$$

式中 x 表示原始信号, y 表示 IMF 分量信号, ESK 为包络谱峭度, $\rho(x, y)$ 为分量信号与原始信号的相关系数, $E[\cdot]$ 表示数学期望, 原始信号和分量信号的均值分别是 u_x 和 u_y , σ_x 对应原始信号而 σ_y 对应分量信号的标准差, env_y 表示分量信号的包络谱, u_{env_y} 表示分

量信号包络谱均值。

本文引入加权包络谱峭度（WESK）作为筛选包含主要故障信息 IMF 的依据。该指标不仅综合考虑了故障周期性冲击对 IMF 形态特征的影响，同时兼顾了 IMF 与原始信号之间的相关性。因此，以 WESK 作为敏感分量的选取准则，能够在最大程度上避免故障信息的遗漏。选取 WESK 值较大的 IMF 分量并予以保留，再将其进行累加，累加后的信号即可作为提取轴承故障特征的有效表征。

基于上述方法，首先，我们以源域数据集中 OR 的振动信号为例，使用 TVFEMD 对故障信号进行分解，并计算每个 IMF 分量的加权包络谱峭度(WESK)，结果如图 4.7(a)所示。接下来选择 WESK 值最大于所有 IMF 分量平均值的 IMF 分量进行信号重构，这些 IMF 分量包括 IMF1、IMF2 和 IMF3。重构后的信号见图 4.7(b)。对重构信号进行包络谱分析，结果如图 4.7(c)所示。

由图 4.7(b)和图 4.7(c)可以看出，基于 TVFEMD 重构后，在一定程度上改善了故障信号的包络谱特征，可以提取到一部分的故障特征频率。然而在重构信号的包络谱中，也仅能识别提取到故障特征频率的 6 倍频，而且包络谱中干扰频率的幅值也较大，总体来讲，并没有获得满意的故障特征频率提取效果。

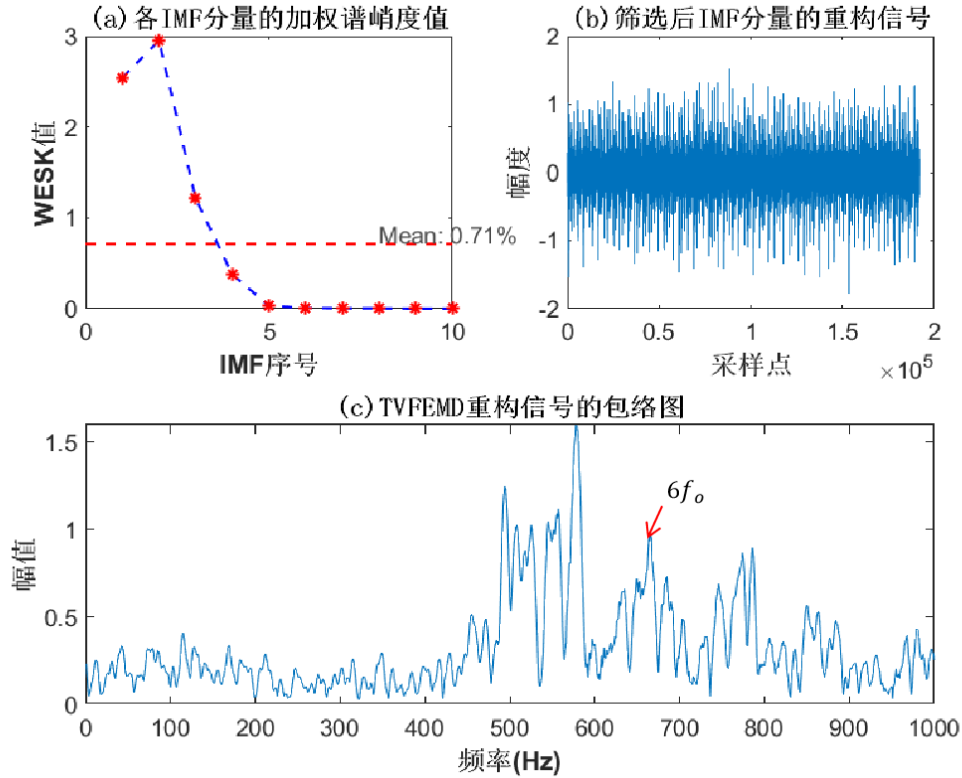


图 4.7 IMF 分量的选择和用所选 IMF 分量的重构

(3) 自适应 MOMEDA 算法

对 TVFEMD 分解筛选后的 IMF 分量进行累加后仍含高频噪声，因此需进一步降噪以突出滚动轴承的故障冲击特征。MOMEDA^[4]是一种有效的非迭代降噪方法，但其关键参数故障周期 T 和滤波器长度 L 主要通过人为主观经验而定，限制了应用效果。为此，本文采用融合冲击指标（SII）作为目标函数，结合自相关极大值和包络谱峭度评价指标，利用灰狼算法自适应寻找参数 T 和 L 的最优组合，以实现最优降噪滤波。

设冲击信号采集传感器获得的信号为

$$x = Hy + e \quad (18)$$

在上述公式中，传递函数用 H 表示， y 表示采集到的故障脉冲序列。假设 x 为传感器采集到的振动信号， e 为采集信号中噪声部分。MOMEDA 算法的核心目标是滤除信号中的噪声部分 e ，从而得到未被污染的故障振动信号，即：

$$y = fx = \sum_{k=1}^{N-L} f_k x_{k+L-1} \quad (19)$$

MOMEDA 算法提出多点 D -范数^[5]，即

$$D(y, t) = \frac{1}{\|t\|} \frac{t^T y}{\|y\|} \quad (20)$$

其中， t 表示待求解的目标权重向量。求解 t 的过程也即为求解多点 D -范数最大化的问题，可表示为：

$$\max_f D(y, t) = \max_f \frac{t^T y}{\|y\|} \quad (21)$$

对式(21)求解问题等价于解方程

$$\frac{d}{df} \left(\frac{t^T y}{\|y\|} \right) = 0 \quad (22)$$

式中， $f = (f_1, f_2, \dots, f_L)$ ， $t = (t_1, t_2, \dots, t_{N-L})$ 。由式(19)-式(22)式可推得

$$\frac{d}{df} \left(\frac{t^T y}{\|y\|} \right) = \|y\|^{-1} (t_1 M_1 + t_2 M_2 + \dots + t_k M_k) - \|y\|^{-3} t^T y X_0 y = 0 \quad (23)$$

式中， $k = 1, 2, \dots, N - L$ 。

令 $X_0 = [M_1, M_2, \dots, M_k]$ ，则式(23)进一步简化为

$$\begin{cases} \|y\|^{-1} X_0 t - \|y\|^{-3} t^T y X_0 y = 0 \\ \frac{t^T y}{\|y\|^2} X_0 y = X_0 t \end{cases} \quad (24)$$

将 $y = X_0^T f$ 代入式(24)中，取其特解记为

$$f = (X_0 X_0^T)^{-1} X_0 t \quad (25)$$

将式(26)中的结果作为滤波器系数，代入到式(19)中，即可恢复原始故障信号 $y = X_0^T f = X_0^T (X_0 X_0^T)^{-1} X_0 t$ 。

在 MOMEDA 的使用过程中，需要输入窗口函数参数 w 、滤波器长度 L 以及故障周期 T ，参数的正确选择是 MOMEDA 滤波噪声的前提。

根据上述，首先构建一个新的融合指标 SII，将其作为目标函数。然后利用 GWO 算法寻优，自动选取 MOMEDA 最佳参数组合 $[T_s, L]$ 。参数自适应 MOMEDA 算法在轴承微弱故障诊断中的具体实现步骤如下：

步骤（1）：利用 TVFEMD 对轴承故障振动信号进行分解，通过加权包络谱峭度 WESK 选取包含故障信息较多的 IMF 分量，对选取后的 IMF 分量进行累加作为初始信号；

步骤（2）：选取 $w=2048$ 的矩形窗作为窗函数；

步骤（3）：灰狼优化算法的种群规模一般在 10~50 之间。本章设定狼群种群规模为

20, 在寻优范围内随机产生 20 个位置作为狼群初始位置, 并设定最大迭代次数为 20。

步骤 (4): 计算每个狼在当前位置下的适应度函数值, 并保存适应度值最好的前 3 匹狼作为狼, 并更新灰狼位置。

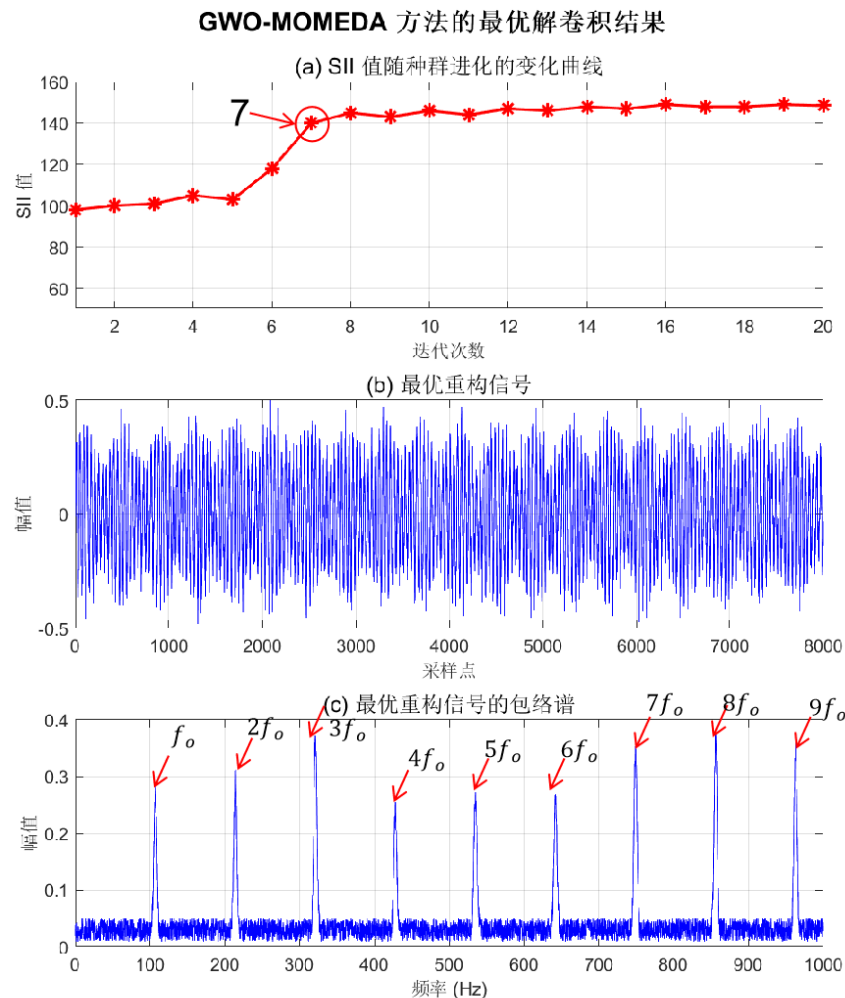


图 4.8 外圈故障 TVFEMD-GWO-MOMEDA 方法的分析结果

经 TVFEMD 重构信号并结合最优参数的 MOMEDA 降噪处理后, 得到的时域信号与其包络谱分别如图 4.8(b)与图 4.8(c)所示。结果表明, 原本被强噪声与干扰掩盖的周期性冲击特征在一系列处理后得以清晰显现, 同时大部分随机噪声和无关干扰成分被有效抑制。在包络谱中, 外圈故障特征频率 f_0 及其倍频位置均出现显著谱线, 验证了外圈故障冲击的存在。该诊断结果与实验前的故障设定完全一致, 说明所提出方法能够准确识别轴承外圈故障特征。

4.5 特征提取

滚动轴承的振动信号中蕴含着大量反映其健康状态的有效信息, 若选用合适的特征提取方法, 便能有效挖掘出潜在的故障特征, 并将其作为健康状态识别的重要依据。常见的特征提取途径主要包括时域、频域和熵域分析, 因此本节将对这三类方法的典型特征指标

进行阐述。

4.5.1 时域特征

时域特征指标是一种振动加速度信号在时域幅值上的统计特征，通常情况下时域特征指标可以划分为两个种类：有量纲指标与无量纲指标^[9]。通常情况下，有量纲特征能随着故障加深而数值增大，能直接表征故障严重程度，但是对工况敏感，易受外部运行条件的干扰。无量纲特征通常与信号的概率密度函数形态有关，可以反映信号的统计分布特性。

设振动信号的离散化形式为 $x_i (i = 1, 2, \dots, N)$ ，时域特征的详细计算公式及特征含义如表 4-5 和表 4-6 所示。

表 4-5 振动信号的有量纲时域特征

特征	计算公式	特征含义
峰值	$T_{peak} = \frac{1}{2} [\max(x_i) - \min(x_i)]$	表征振动信号振动的剧烈程度，对早期故障信号较敏感
均值幅值	$T_{mean} = \frac{1}{N} \sum_{i=1}^N x_i$	振动信号的一阶矩，表征信号整体平均强度
均方根值	$T_{rms} = \sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2}$	振动信号的二阶矩，描述信号的振动能量，表征轴承的总体劣化程度，对磨损类故障有效
方根幅值	$T_r = \left(\frac{1}{N} \sum_{i=1}^N \sqrt{ x_i } \right)^2$	表征振动信号的振动强度
绝对平方值	$T_{A-mean} = \frac{1}{N} \sum_{i=1}^N x_i $	表征振动信号的整体强度
方差	$T_{var} = \frac{1}{N-1} \sum_{i=1}^N (x_i - T_{mean})^2$	振动信号的二阶中心距，描述信号的振动能量，表征信号总体偏离中心的波动程度
标准差	$T_{std} = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (x_i - T_{mean})^2}$	与方差类似，表征信号的总体波动程度

表 4-6 振动信号的无量纲时域特征

特征	计算公式	特征含义
峰值因子	$T_p = \frac{T_{peak}}{T_{rms}}$	表征振动信号中是否存在冲击分量
波形因子	$T_s = \frac{T_{rms}}{T_{A-mean}}$	表征振动信号的波形变化

脉冲因子	$T_I = \frac{T_{peak}}{T_{A-mean}}$	表征振动信号中是否存在异常脉冲
裕度因子	$T_{mar} = \frac{T_{peak}}{T_r}$	表征轴承的磨损情况
偏度因子	$T_{ske} = \frac{\frac{1}{N-1} \sum_{i=1}^N (x_i - T_{mean})^3}{T_{std}^3}$	表征振动信号的分布对称性的统计量
峭度因子	$T_{kur} = \frac{\frac{1}{N-1} \sum_{i=1}^N (x_i - T_{mean})^4}{T_{std}^4}$	表征振动信号波形平缓程度，通过反映振动信号的分布特征来表征冲击特性

时域特征指标对于故障特征信息表征能力，一方面，时域特征指标计算效率高且时效性较好；而另一方面，时域特征指标的表征能力较为有限，且极易受到噪声影响。因此，为了提高轴承故障诊断结果的可信度，通常需要联合频域、熵域等特征进行综合分析，以提高诊断的准确度。

4.5.2 频域特征

频域特征指标是通过对振动信号进行傅里叶变换，将其由时域转换至频域后提取的一类特征参数。该类指标能够揭示信号在频率维度上的能量分布与谱结构特征，从而为滚动轴承故障诊断提供重要依据。常用的频域指标包括频谱均值、频谱方差、频谱峭度、频谱偏度等统计量，分别反映频率分布的集中趋势、离散程度及谱形态特征。此外，频域能量比、主频率及频带能量分布等指标能够刻画信号能量在不同频率区间的分布规律，对识别故障特征频率及其倍频成分具有重要意义。相比时域分析，频域特征能够更直观地反映出由局部损伤所引起的周期性冲击特征，因而在滚动轴承故障的特征提取与模式识别中得到了广泛应用。

表 4-7 振动信号的频域特征

特征指标	计算公式	特征含义
均值幅值频率	$F_{mean} = \frac{1}{M} \sum_{j=1}^M y_j$	表征振动信号在频域内的能量大小
重心频率	$F_{cen} = \frac{\sum_{j=1}^M f_j \times y_j}{\sum_{j=1}^M y_j}$	表征振动信号在频域的中心位置

标准差频率	$F_{std} = \frac{\sqrt{\sum_{j=1}^M (f_j - F_{cen})^2 \times y_j}}{\sum_{j=1}^M y_j}$	表征振动信号在频域中总体偏离中心的波动程度
均方根频率	$F_{rms} = \sqrt{\frac{\sum_{j=1}^M f_j^2 \times y_j}{\sum_{j=1}^M y_j}}$	表征振动信号在频域中主频带的位置变化
偏斜度频率	$F_{ske} = \frac{\sum_{j=1}^M (f_j - F_{cen})^3 \times y_j}{M \times F_{std}^3}$	表征振动信号在频域中向重心频率靠近的集中程度
峭度频率	$F_{kur} = \frac{\sum_{j=1}^M (f_j - F_{cen})^4 \times y_j}{M \times F_{std}^4}$	表征振动信号在频域中的集中或分散的程度

4.5.3 熵域特征

熵域特征指标是一类基于信息论与非线性动力学理论的特征量，主要用于衡量振动信号复杂性与不确定性。在滚动轴承故障诊断中，熵类指标能够有效揭示信号的随机性、混沌性及非平稳特征，对弱故障特征的提取具有较高敏感性。常用的熵类特征包括近似熵与样本熵，用于刻画时间序列模式的复杂程度；排列熵则通过分析相空间重构后的符号序列分布来反映系统动态演化特性；此外，能量熵、谱熵及模糊熵等指标能够从能量分布、频谱特性及模糊集理论等不同角度揭示信号的不确定性。总体而言，熵域特征能够从信息量和复杂性层面对滚动轴承运行状态进行有效表征，尤其适用于强噪声及非平稳环境下的故障诊断。

4.5.4 基于 MixHop 模型构建特征矩阵

(1) 特征矩阵构建

滚动轴承的原始振动信号本质上是一维时间序列，为了充分挖掘其中蕴含的健康状态信息，需要对多种故障特征进行提取。为此，本文通过联合提取时域特征、频域特征及熵域特征，拼接特征构建特征矩阵，实现对滚动轴承故障信息的全面表征与综合分析。

设原始振动信号为 $X = (x_1, x_2, \dots, x_n)$ ， n 为信号长度，基于 TVFEMD 与自适应 MOMEDA 算法的特征矩阵具体构建流程如下：

1) 将原始信号 $X = (x_1, x_2, \dots, x_n)$ 进行 TVFEMD 分解，并根据 4.2.2 中所提的 WESK 对分解后的信号进行最优 IMF 组合筛选。

2) 通过自适应 MOMEDA 算法，构建融合冲击指标 SII，利用 GWO 算法找到最佳参数组合，对步骤 1) 中分量信号组合进行二次降噪处理，提取最优的 IMF 组合信号 $U = (u_1, u_2, \dots, u_n)$ 。

3) 将最优 IMF 组合信号 $U = (u_1, u_2, \dots, u_n)$ 等分为 m 个长度为 M 的窗口 ω_i , $i = 1, 2, \dots, m$ 。

4) 分别从 ω_i 中提取 16 个时域特征指标 $f_i^p (p = 1, 2, \dots, 16)$, 从 Y_i 中提取 13 个频域特征指标 $f_i^q (q = 17, 18, \dots, 29)$, 并从 X_i 中提取 7 个熵域特征指标 $f_i^l (l = 30, 31, \dots, 36)$, 构建成特征矩阵 X , 具体特征如表 4-8 所示。

表 4-8 振动信号特征名称列表

序号	特征名称	序号	特征名称	序号	特征名称
1	均值	13	脉冲指标	25	频域幅值峭度指标
2	方差	14	裕度指标	26	频率标准差
3	方根幅值	15	偏斜度指标	27	频域频率歪度
4	绝对平均幅值	16	峭度指标	28	频域频率峭度
5	最大值	17	频域幅值平均值	29	平方根比率
6	最小值	18	重心频率	30	近似熵
7	峰值	19	均方频率	31	包络熵
8	有效值	20	频率方差	32	EMD 能量熵
9	偏斜度	21	均方根频率	33	功率谱熵
10	峭度	22	频域幅值方差	34	样本熵
11	峰值指标	23	频域绝对平均幅值	35	小波包能量熵
12	波形指标	24	频域幅值偏度指标	36	模糊熵

(2) MixHop 模型原理

MiXHop (Mixing Multiple Orders of Graph Convolutions) 模型^[10]是一种图神经网络 (GNN) 模型, 其核心思想是在单层中同时聚合节点的多阶邻居信息。传统的图卷积网络 (GCN) 通常只聚合一阶邻居的信息, 而 MiXHop 通过引入多阶邻接矩阵的幂次, 实现了在同一层中同时学习一阶、二阶乃至更高阶邻居的特征。随后, 这些不同阶的特征被串联在一起, 用以丰富节点表示。其基本流程如下:

(i) 输入特征: 将特征矩阵 $X \in \mathbb{R}^{N \times d}$ 输入模型, 其中 N 为窗口数, d 为特征维度。

(ii) 多阶邻居扩散：利用邻接矩阵的不同幂次($A, A^2, A^3, \dots$)对输入特征进行传播，从而在同一层内同时捕获一阶、二阶及更高阶邻居的特征信息。

(iii) 特征映射与拼接：对每阶邻居特征分别进行线性变换，并在特征维度上拼接，得到包含多阶信息的综合特征表示。

(iv) 非线性与正则化：对拼接后的结果施加非线性激活函数（如 ReLU），并结合 Dropout 等正则化方法，提升模型的表达能力与泛化性能。

经过 MixHop 模型生成的多特征融合矩阵如下图 4.9 所示。四个子图分别对应不同类别轴承工况在 MixHop 模型特征融合后得到的特征矩阵可视化结果。通过观察可以清晰地看出各子图之间呈现出明显的空间分布差异。例如(a)与(c)的特征矩阵中存在较多分布均匀的高低频交织纹理，而(b)的特征分布较为平滑，整体幅值集中在较低范围；(d)则在部分局部区域表现出较强的块状高值特征。这表明 MixHop 模型在聚合多阶邻居信息后，能够有效提取不同工况下样本具有判别性的特征，使不同故障类别之间在特征空间上呈现出差异化分布。

对比四个子图可以发现，不同类别的矩阵在颜色分布模式和局部纹理特征上差异显著。这意味着经过 MixHop 特征融合后的矩阵具备较好的类别可分性，有助于后续分类器实现对不同轴承故障状态的准确区分。

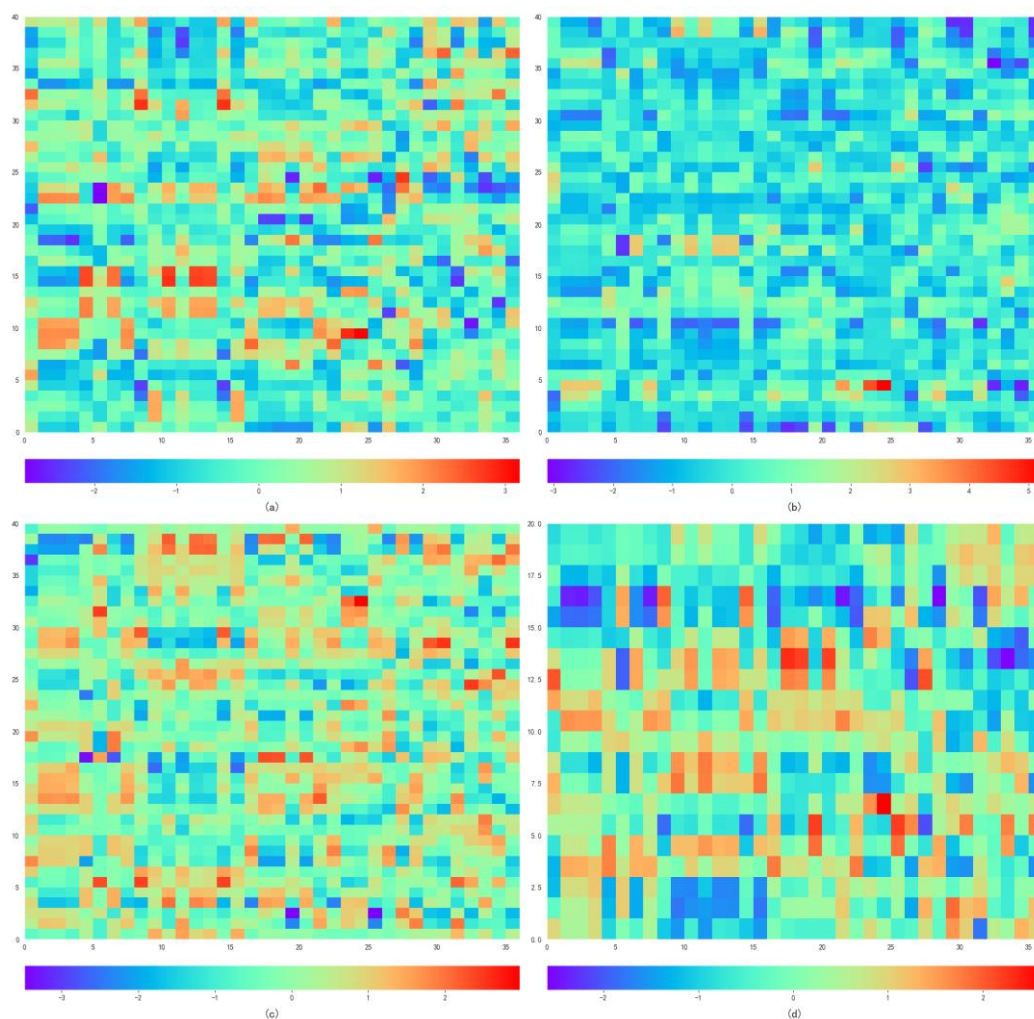


图 4.9 多特征融合矩阵图

5 问题 2：基于 Vision Transformer 的轴承故障诊断模型构建

5.1 问题分析

根据题意，需要根据问题 1 中基于 MixHop 模型提取的多特征融合故障特征基础上，构建轴承诊断模型，并使用该模型对问题 1 中提取的源域数据进行预测。本文对该问题的解答可以具体分为三个步骤：

(1) 对问题 1 筛选的特征数据集进行 8:2 的随机划分，前者用于算法的训练，后者用于验证该算法的的准确率。

(2) 在多种模型上进行训练，对测试集进行预测并记录结果。

(3) 联合多种评估指标，选择精度最优的模型，并作为轴承故障诊断模型。

具体流程如图 5.1 所示，

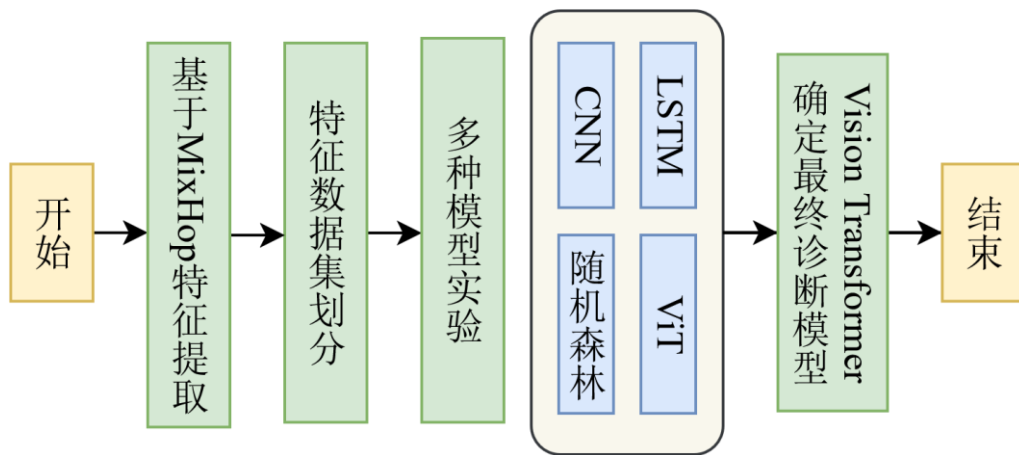


图 5.1 问题 2 思路流程图

5.2 模型建立

5.2.1 Vision Transformer (ViT)

Vision Transformer 是首个成功将 Transformer 架构应用于计算机视觉的研究工作，创新性地证明了该架构不仅在自然语言处理任务中具有优异表现，在图像分类等视觉任务中同样能够取得显著性能提升，其结构如图 5.2 所示。

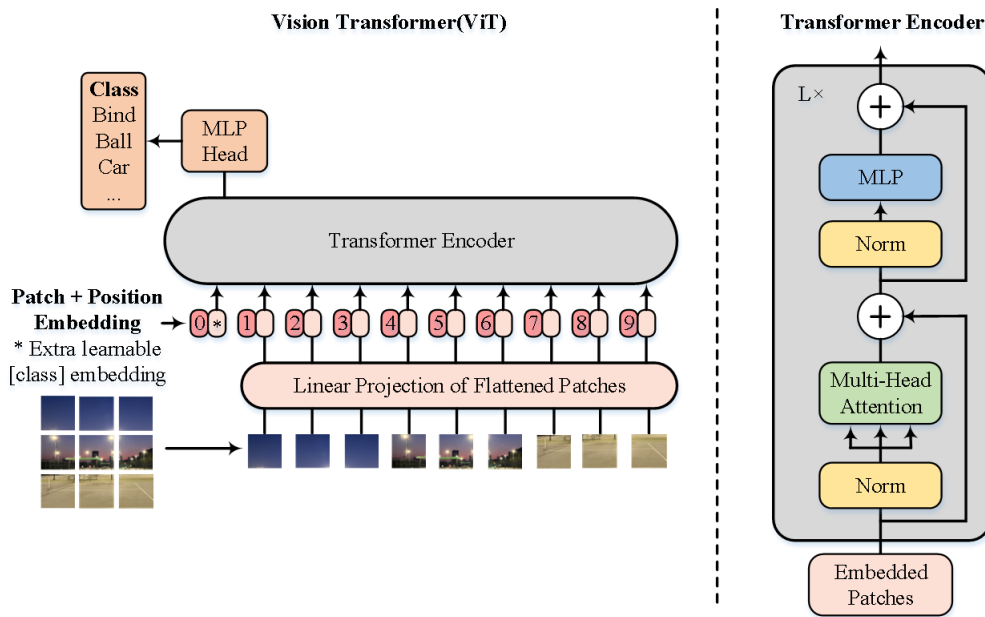


图 5.2 ViT 结构图

Vision Transformer 处理图像的整体流程如下：

(1)图像分块:将输入图像分割成一系列固定大小且互不重叠的 Patch,通常选择 16×16

像素的 Patch 作为分割单位。每个这样的 Patch 随后被展平为一维向量，这样，原始图像被转化为一个由一维向量构成的序列。其形式与自然语言中的词序列类似，使得图像数据得以类似于文本数据的方式被模型处理。

(2)线性投影:在将图像分割成一系列一维向量后，下一步是对这些向量执行线性投影，即进行图像的 Token 化处理，以实现维度的变换。

(3)类别 Token 引入:为了便于后续分类任务，ViT 在序列前添加一个额外的类别 Token（Class Token）。这个类别 Token 并非直接从输入图像中提取的特征，而是一个可学习的参数，在模型训练过程中不断优化，以适应不同的图像数据和分类任务。

(4)位置编码:由于 Transformer 模型无法直接处理序列的位置信息，ViT 还引入了位置编码。通过将位置编码与 Token 序列相加，模型得以在后续的计算中充分利用每个 Token 的位置信息。

(5)Transformer 编码器处理:结合了位置编码的 Token 序列被输入到 Transformer 的编码器中进行处理。编码器中的多头自注意力机制负责在 Patch 之间进行全局特征提取，这使得模型能够捕捉到图像中不同区域之间的依赖关系。同时，编码器中的前馈神经网络则对各个 Patch 的特征进行进一步的变换和增强，从而生成更具表达力的特征表示

(6)分类输出:从编码器最后一层输出序列中提取类别 Token 对应的表示，被用于后续的多层感知机进行分类处理。

5.2.2 卷积神经网络 (Convolutional Neural Network, CNN)

CNN 在图像处理领域取得了巨大成功，其核心在于利用卷积核自动学习局部特征。在故障诊断中，CNN 常被用于处理一维时间序列信号或二维时频图。本研究中，我们将 MixHop 优化后的特征矩阵视为二维数据（或重塑为适合一维卷积的形式），输入到多层卷积网络中。CNN 模型通常包含交替的卷积层、激活层、池化层，用于提取特征层次结构，末端连接全连接层进行分类。其优势在于能够捕获特征中的局部模式和空间不变性。CNN 模型流程如图 5.3 所示。

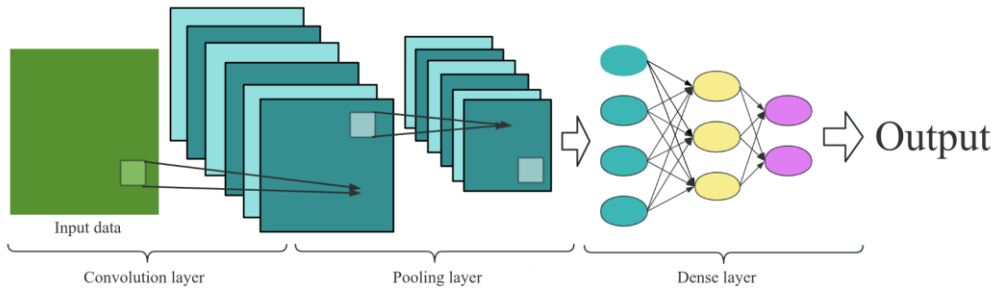


图 5.3 CNN 流程图

5.2.3 长短期记忆网络(Long Short-Term Memory, LSTM)

LSTM 是一种特殊的循环神经网络 (RNN)，专门设计用于处理和预测时间序列数据。它通过引入门控机制（输入门、遗忘门、输出门）来有效解决传统 RNN 中的梯度消失和梯度爆炸问题，使其能够学习长期依赖关系。对于本研究中的特征矩阵，每个样本的 MixHop 优化特征向量可以被视为一个时间步，或者将多个样本按某种顺序排列形成一个序列，输入到 LSTM 网络中。LSTM 能够捕获故障特征序列中的时序动态和上下文信息，这对于理解故障演变过程可能至关重要。LSTM 模型结构如错误!未找到引用源。所示。

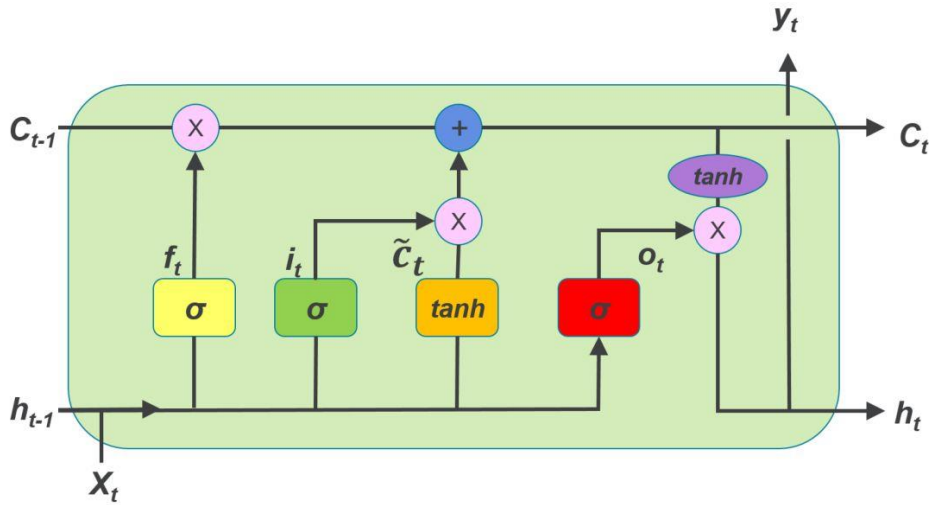


图 5.4 LSTM 结构图

5.2.4 随机森林(Random Forest, RF)

随机森林是一种集成学习方法，由多棵决策树组成。每棵决策树在训练时从原始数据集中有放回地随机采样 (Bootstrap aggregating)，并随机选择特征子集进行节点分裂。最终的分类结果由所有决策树的投票决定。RF 模型具有对高维数据处理能力强、不易过拟合、对噪声和异常值不敏感等优点，在机器学习领域被广泛应用于分类和回归任务。在本研究中，RF 模型将直接以 MixHop 优化后的特征矩阵作为输入，对每个样本进行故障类型分类。随机森林算法流程图如 5.5 所示。

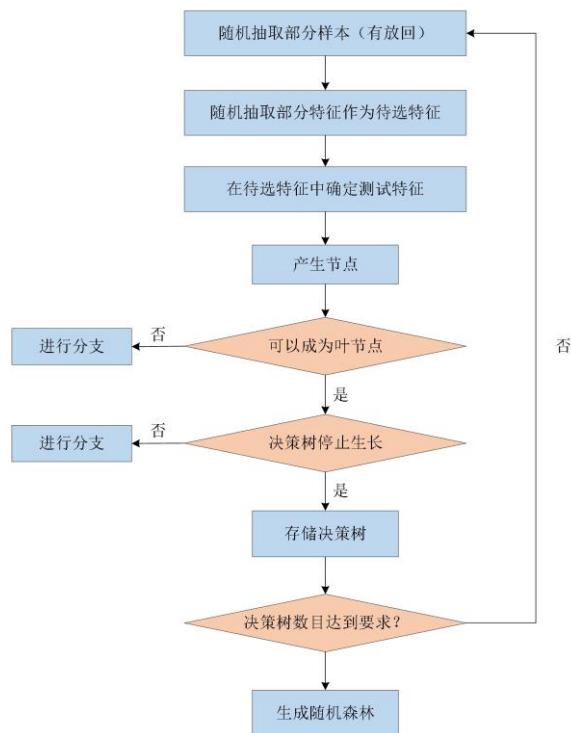


图 5.5 随机森林算法流程图

5.3 模型求解

5.3.1 模型参数设置

模型名称	变量名称	参数值
Vision Transformer	Patch 大小	16
	嵌入维度	128
	Transformer 编码器层数	12
	注意力头数	12
	MLP 隐藏层比例	4.0
	Dropout 率	0.1
CNN	第一层卷积滤波器数量	32
	卷积核大小	(3,3)
	步长	(1,1)
	填充方式	same
	池化核大小	(2,2)
	激活函数	ReLU
	Dropout 率	0.25

	全连接层神经元数	128
LSTM	LSTM 层神经元数	128
	Dropout 率	0.2
	循环 Dropout 率	0.2
	时间步长	30
	输入特征数	10
	批量大小	32
	训练轮数	100
RF	弱分类器的个数	1000
	子树评估标准	Gini
	树的最大深度	20
	叶子节点最少样本数	1

5.3.2 数据集准备

根据问题一的结果，目前的训练数据集包含 112 条样本，每条样本有 40 个窗口，每个窗口里面有 36 维的特征。为了评估模型性能，本文采用随机划分的方式，将特征数据集拆分成训练集以及测试集。模型在训练集上进行训练，并在测试集上验证评价指标，并选择最终的预测模型。

5.3.3 评价指标

在实验指标方面，本文使用二分类精度（Binary Classification Accuracy, Acc）以及曲线下面积（Area Under Curve, AUC）对算法进行评估。Acc 度量的是正确预测的数量占总预测数的比，如式（26）所示：

$$Acc = \frac{TP + TN}{TP + TN + FP + FN} \quad (26)$$

式中， TP (True Positive)表示正确预测的真样本， TN (True Negative)表示正确预测的假样本， FP (False Positive)表示错误预测的假样本， FN (False Negative)表示错误预测的真样本。AUC 是比 Acc 更加综合且鲁棒的评价指标，其定义是受试者工作特征曲线（Receiver Operating Characteristic Curve, ROC）下的面积，ROC 曲线描述的是分类阈值变化时，真阳性率 TPR (True Positive Rate)和伪阳性率 FPR (False Positive Rate)的变化，可以通过式（27）与式（28）计算得到

$$TRP = \frac{TP}{TP + FN} \quad (27)$$

$$FPR = \frac{FP}{FP + TN} \quad (28)$$

5.3.4 模型性能分析

四个模型准确率如表 5-1 所示，Vision Transformer 模型取得了最高的准确率。

表 5-1 模型准确率

模型	准确率
ViT	95.7%
LSTM	87.0%
CNN	82.6%
随机森林	87.0%

混淆矩阵对比图、模型损失对比图以及 ROC 对比图，如图 5.6、图 5.7 和图 5.8 所示。从四种模型的混淆矩阵对比可以看出，Vision Transformer 在本实验设置下对轴承故障类别区分能力最强（总体准确率与各类召回率最高），随机森林作为基线也表现稳健，而传统的 CNN 与 LSTM 在当前输入表示与超参数配置下表现较差。误分主要集中在少数具有相似时频特征的类别之间（例如类 2 内圈故障，类 3 外圈故障）

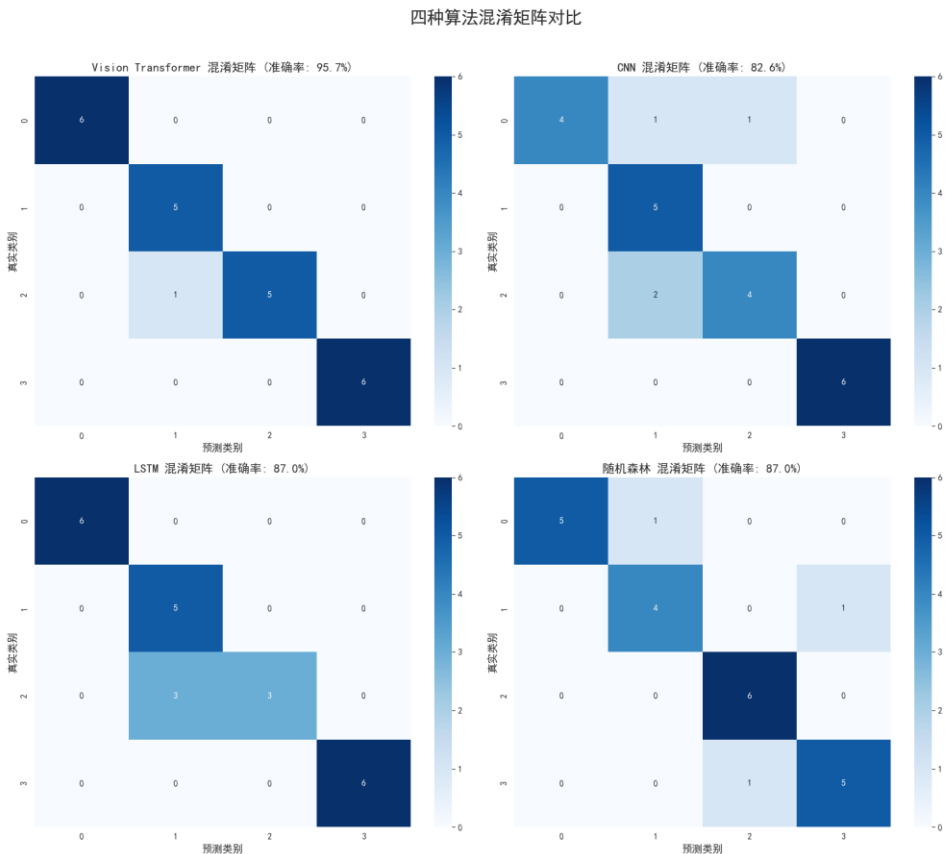


图 5.6 四种算法混淆矩阵对比图

在模型损失对比图中，比较了 LSTM、ViT 和 CNN 三种模型在 0-100 个 epoch 的训练损失（Train Loss）随训练轮次的变化曲线。可以清晰看出三条曲线的总体行为差异明显：ViT 模型在前几轮损失急剧下降，并在大约第 5-10 个 epoch 之后降到接近零且保持平稳；LSTM 模型在损失下降速度中等，前期快速下降后仍继续缓慢收敛，在大约 40-60 个 epoch 逐步接近零；CNN 模型的损失基本维持在较高水平（约 1.2 左右）并缓慢、微幅度下降，未出现明显收敛到低损失的趋势。

图 5.8 展示了 Vision Transformer、LSTM 和 CNN 三种模型在故障诊断任务中的 ROC 曲线表现。灰色虚线为随机分类器的基准线(AUC=0.5)，三种模型均显著高于该基准，说明都具备较好的判别能力。ViT 的 AUC=0.953，曲线最靠近左上角，表示其对正负样本的区分能力最强。在低 FPR 区域(0-0.2)，ViT 的 TPR 提升最快，说明其在严格控制误报的条件下仍能保持较高的检出率，适合高安全性场景。

综合三张图可以得出结论：我们采用的 ViT 模型对于轴承故障诊断效果最好，既能快速收敛又能保持高分类准确率，能够有效的识别故障类型

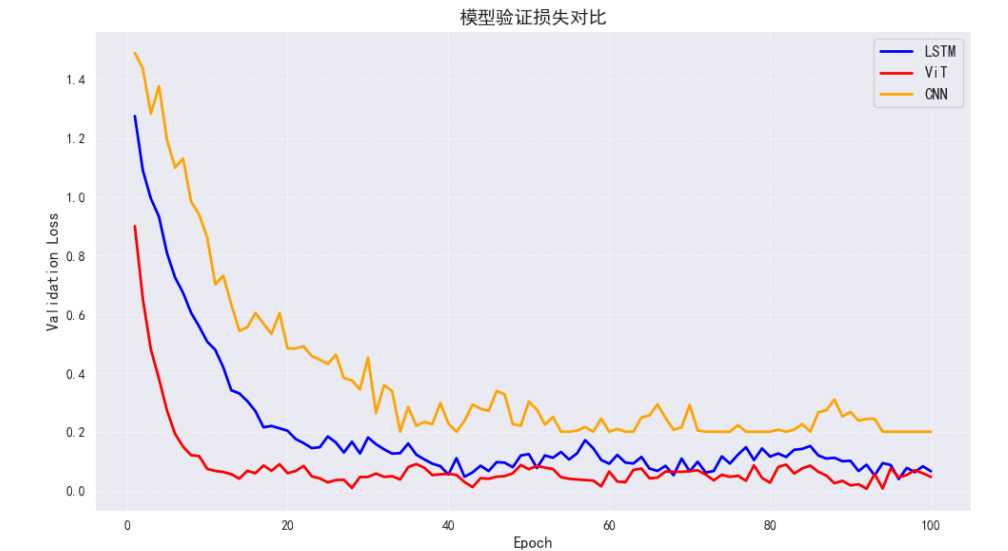


图 5.7 模型损失对比图

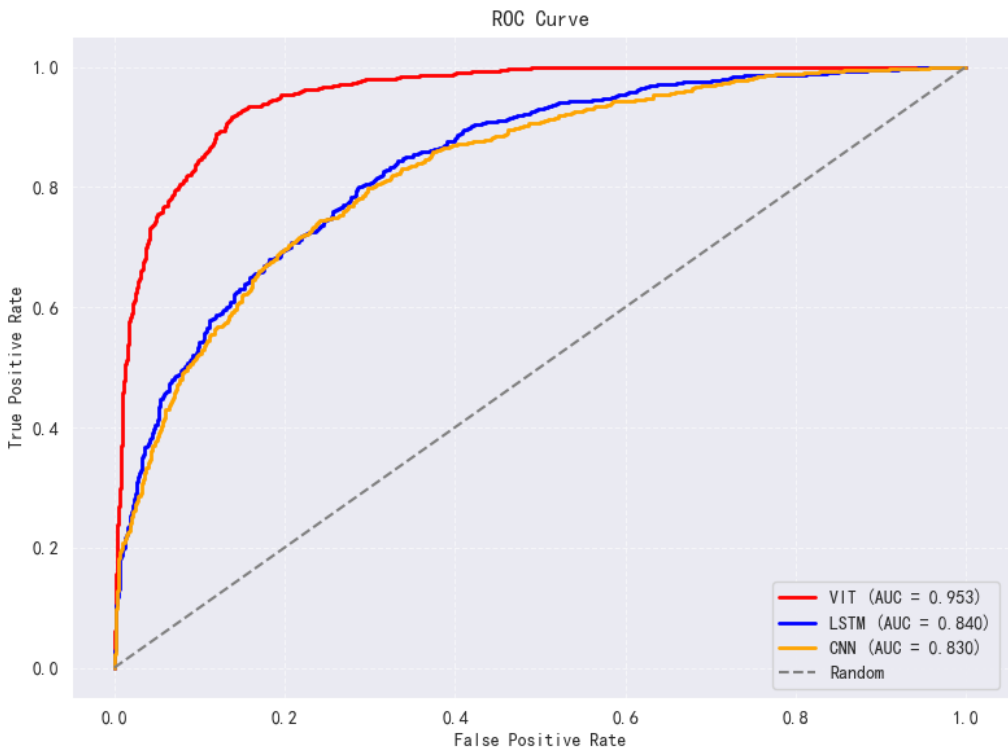


图 5.8 ROC 对比图

5.3.5 模型稳定性分析

由于每次训练/测试数据的拆分不同，且加之基于学习的方法的随机性。算法表现的稳定性也是需要考量的重要因素之一，如图 5.9 所示，本文展示了四种算法测试 10 次的结果。结果以箱线图的形式表现。其中，四种颜色的矩形表示第 25% 的误差值与第 75% 的误差值，越宽表明其算法随机波动性越强。其中箱外的黑色实线表示误差的合理上下界，超出该范围说明此次实验的误差不属于该模型的实际表现范围。总体而言，基于 Vision Transformer 的方法在稳定性方面也具有更高水平。

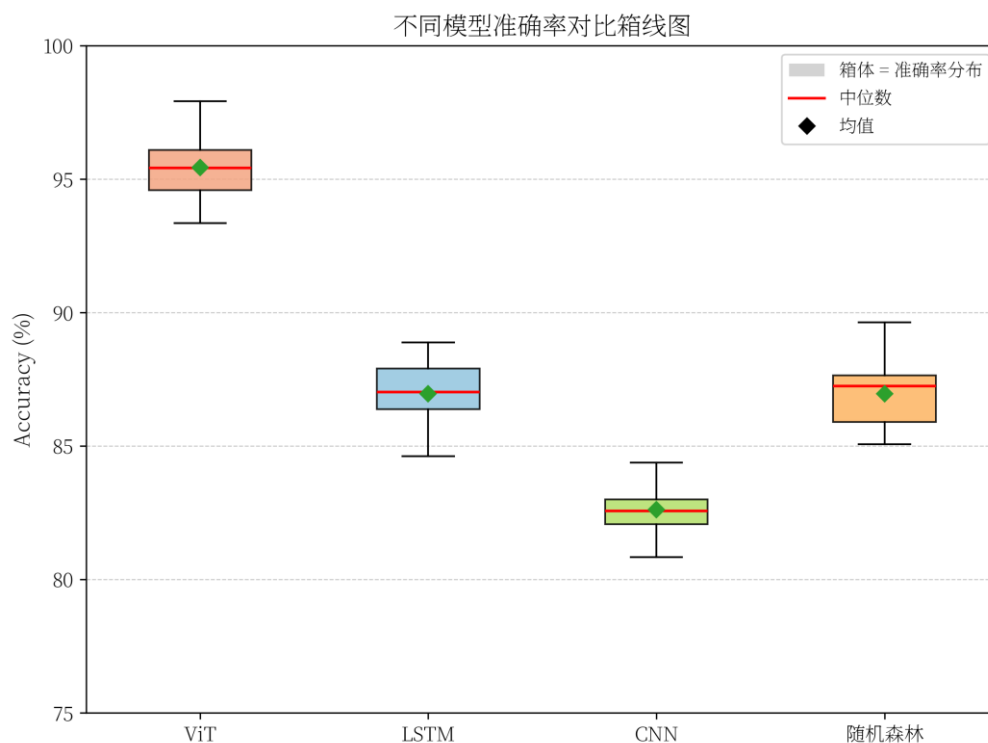


图 5.9 不同模型预测稳定性对比箱线图

6 问题 3：基于 Vision Transformer 的领域自适应无标签故障诊断

6.1 问题分析

根据题意，在任务 2 设计的基于 Vision Transformer 诊断模型框架下，设计针对性的迁移学习方法，构建适用于目标域的诊断模型，对未知标签数据进行高效分类与标定，具有重要工程意义。

本题对该问题的解答可具体分为以下三个步骤：

- (1) 通过 KDE 图对源域与目标域的特征分布进行对比分析，发现两者存在显著偏差。
- (2) 基于最大均值差异 (MMD) 和统计特征变换的度量方法，提出不同数据空间、不同阶数下的二阶矩与高阶矩联合匹配策略，用于提取多重域不变特征，从而有效缓解分布偏差。
- (3) 构建基于 Vision Transformer 的无监督领域自适应网络，通过高阶矩与二阶矩的联合匹配，缩小源域与目标域的分布差异，并借助权值共享机制辅助源域与目标域网络提取可迁移特征，实现高效领域自适应。

具体流程如图 6.1 所示。

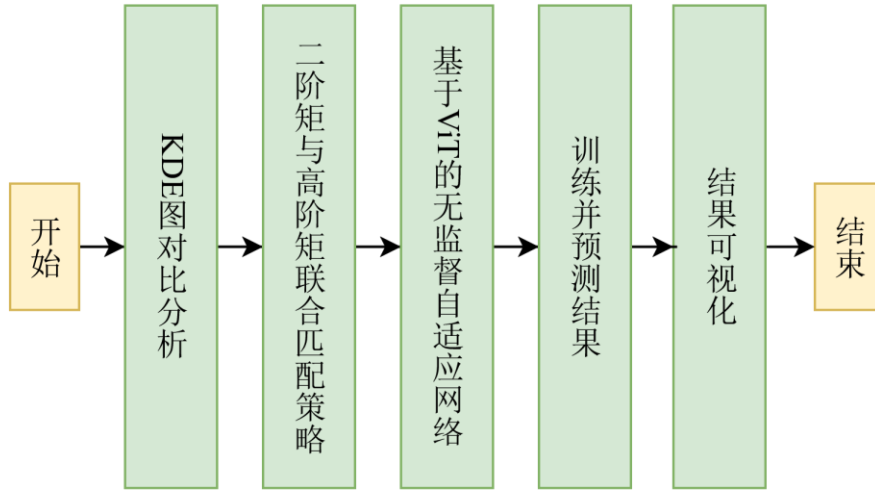


图 6.1 问题 3 思路流程图

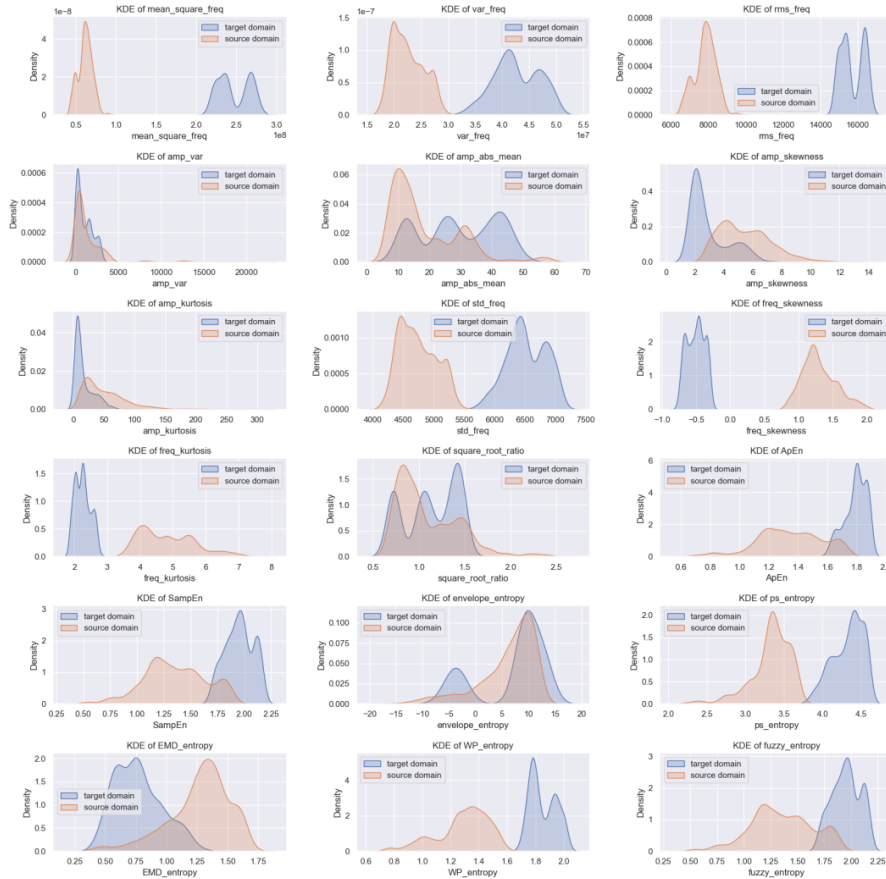


图 6.2 源域和目标域的部分特征分布图

6.2 融合高阶与二阶矩匹配的无监督深度领域适应网络

6.2.1 分布差异度量

通过查看源域和目标域的 KDE 图可知，问题一提取的大部分特征在两个域都有显著分布差异。因此，考虑领域自适应方法，即通过度量并减小领域之间的特征分布距离来提取域不变的特征，从而实现迁移学习的目的。好的距离指标有助于领域自适应过程中

提取更鲁棒的迁移特征，并建立更高效的迁移学习模型。领域相似性度量的思路之一是分别对不同的概率分布进行建模，应用较为广泛的两种方法包括：

(1) 最大均值差异

最大均值差异 (Maximum Mean Discrepancy, MMD) 计算两个概率分布映射到其他数据空间后的均值之差。通常将分布映射到由显著核 K 定义的可再生核 希尔伯特空间 (Reproducing Kernel Hilbert Space, RKHS) 中，则两个概率分布之间 MMD 的平方等价于在此空间中分布的平均距离

$$MMD_{\mathcal{H}_k}^2(p, q) := \|\mathbb{E}_{x \sim p}[\phi(x)] - \mathbb{E}_{x \sim q}[\phi(x)]\|_{\mathcal{H}_k}^2 \quad (29)$$

式中， p 和 q 为两个概率分布； $\mathcal{H}$ 代表 RKHS； $\phi(\cdot)$ 为原始空间到 RKHS 的映射函数。

核函数为映射的内积，定义为

$$k(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle \quad (30)$$

式中， $\langle \cdot, \cdot \rangle$ 为内积计算。

(2) 统计特征变换

通过将不同概率分布的数据进行统计特征变换，在变换后的子空间中进行特征分布自适应。统计特征变换中有代表性的 CORAL (CORrelation ALignment) 方法能够学习一个二阶特征变换，使领域间的统计特征距离最小，数学定义为

$$\min_A \|A^T C_{D_1} A - C_{D_2}\|_F^2 \quad (31)$$

式中， C_{D_1} 和 C_{D_2} 为领域特征的协方差矩阵； A 代表二阶特征变换。

6.2.2 高阶与二阶联合矩匹配方法

在数学中，矩是对变量分布和形态特征的具体定量度量方法之一，能够有效地衡量分布的差异性，直观地度量领域之间的距离。矩匹配策略将不同数据分布的矩的差异作为优化目标，从而减小领域差异。通过在深度迁移学习网络中的较深层设置一个域适应层来提取源域和目标域中网络层的输出特征，并利用矩匹配策略对不同领域的特征分布进行对齐。最终通过减少领域之间的分布异实现领域自适应，协助迁移学习网络学习到跨领域的交叉特征。

首先针对本问的数据场景进行形式化定义，在多种工况下的轴承故障诊断任务中，给出一个有标签的源数据集 $X_S = \{x_i\}_{i=1}^n$, $Y_S = \{y_i\}_{i=1}^n$ 和一个无标签的目标 $X_T = \{x_j\}_{j=1}^m$ 数据集。

通过将矩匹配过程设置为待优化目标函数中的正则项，能够在网络训练过程中逐渐减小领域分布差异，实现领域自适应目的。为更高效、准确地优化源域与目标域之间的分布差异，本章基于最大均值差异和统计特征变换的度量方法，提出了不同数据空间、不同阶数下的二阶矩和高阶矩联合匹配策略，用于提取多重域不变特征，从而覆盖更全面的故障信息。两个矩匹配项介绍如下。

(1) 基于最大均值差异的高阶矩匹配

边缘分布描述数据的整体相似程度，条件分布描述每个类别的相似程度，联合分布在形式上由边缘分布和条件分布相加获得。本问假设参与学习的领域服从相同的条件概率分布，即 $P(Y_S|X_S) \neq P(Y_T|X_T)$ ，因此将边缘分布距离近似为领域之间的联合分布距离，通

过基于 MMD 的距离度量方法来实现边缘分布自适应，从而减小数据分布差异，实现迁移学习。

首先，选择高斯核作为 MMD 中的核函数，其数学表示为

$$k(x_i, x_j) = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right) \quad (32)$$

式中， σ 为核函数宽度

然后，通过计算域适应层所对应的源域和目标域数据特征的分布差异，得到领域之间的距离度量值为

$$MMD^2(P(F_s), P(F_t)) = \left\| \frac{1}{n_s} \sum_{i=1}^{n_s} \phi(F_s) - \frac{1}{n_t} \sum_{i=1}^{n_t} \phi(F_t) \right\|_{\mathcal{H}}^2 \quad (33)$$

式中， F 为神经网络第 1 层的输出向量； $\phi(\cdot)$ 为在 RKHS 中的非线性映射函数由上式能够看出，高阶矩匹配方法通过计算两个数据分布映射到再生核希尔伯特空间后的一阶矩，即映射后的数据均值之差来度量领域差异。而对于原始空间中的分布来说，该方法起到了匹配高阶矩的作用。

(2) 基于统计特征变换的二阶矩匹配

通过最小化域适应层中源域与目标域输出向量的二阶统计特征，即计算 CORAL 损失来实现二阶矩匹配过程，数学表达式为

$$L_{CORAL} = \frac{1}{4d^2} \|C_s - C_t\|_F^2 \quad (34)$$

式中， d 为特征维度； C_s 、 C_t 为源域和目标域特征经过域适应层后输出向量的协方差矩阵，具体定义为：

$$C_s = \frac{1}{n_s - 1} \left[(F_s^l)^T F_s^l - \frac{1}{n_s} (\mathbf{1}^T F_s^l)^T (\mathbf{1}^T F_s^l) \right] \quad (35)$$

$$C_t = \frac{1}{n_t - 1} \left[(F_t^l)^T F_t^l - \frac{1}{n_t} (\mathbf{1}^T F_t^l)^T (\mathbf{1}^T F_t^l) \right] \quad (36)$$

能够看出，CORAL 损失能够计算两个领域中的特征在原始空间中的协方差矩阵之间的距离，起到了匹配二阶矩的作用。将上述高阶矩匹配函数与二阶矩匹配函数之和作为域适应损失，并将两项同时加入深度迁移网络目标函数的正则项中，从而实现源域与目标域特征分布矩的联合匹配。

6.2.3 基于 Vision Transformer 的无监督领域自适应网络设计

在任务二中，已经构建了基于 Vision Transformer 的故障诊断模型，并且性能表现优异。因此，选择其作为深度迁移学习模型的主干网络，将最后的分类层替换为适用于轴承故障诊断任务的网络层和参数，提出基于高阶与二阶联合矩匹配的无监督深度迁移学习模型。模型的网络结构由三部分组成：ViT 骨干网络、域适应层和分类层。其中，ViT 骨干网络用于学习高维的故障特征，域适应层将源域和目标域所对应网络较深层的输出向量作为输入，用于后续的域适应损失计算。

网络结构如图 6.3 中的红色框所示，该结构在域适应层中不进行额外的特征变换，将

领域特征向量直接用于联合矩匹配，并且在分类网络中仅设置了一个全连接层，使矩匹配过程更加简洁高效。源域网络与目标域网络的权值共享。



图 6.3 深度迁移学习网络结构图

通过结合损失函数和正则化项，定义深度迁移学习模型的目标函数，从而经过网络的前向计算后得到优化目标。其中，损失函数由有标签源域数据在训练过程中的分类误差表示，用于衡量源域中训练样本的预测值与真实标签之间的差异；正则化项由式和定义的域适应损失表示，目的在于引入领域分布距离的先验知识，优化领域分布差异，并防止过拟合。最终，目标函数的数学表示为：

$$\min_{\theta} \frac{1}{n} \sum_{i=1}^n J(\theta(x_i), y_i) + \lambda (MMD_{\mathcal{H}}^2(\mathcal{D}_s^l, \mathcal{D}_t^l) + L_{CORAL}(\mathcal{D}_s^l, \mathcal{D}_t^l)) \quad (37)$$

式中， θ 为网络优化的目标参数，包括权重和偏差； l 为域适应层； λ 为正则项系数； $J(\cdot)$ 为通过计算交叉熵损失得到的分类误差。

通过损失的反向传播，即在训练中最小化目标函数，不断更新模型参数，使模型预测更加准确，同时实现分类和领域自适应。通过在正则项中设置边缘分布差异和统计特征变换的领域自适应策略，实现高阶与二阶的联合矩匹配，使模型在不同的数据空间中全面地度量并优化领域分布差异，提高特征分布适配效果。

6.3 模型训练

模型训练过程如表 6-1 所示，输入数据包括有标签源域样本和无标签目标域样本。训练过程中将随机梯度下降（Stochastic Gradient Descent, SGD）作为优化器，学习率设置为 0.01，使用 L2 范数正则化防止过拟合，权重衰减设置为 $5e^{-4}$ 。在每个批次中，网络将输入相同数量的源域数据和目标域数据，从而消除领域大小引起的偏差。

另外，为了提高模型的精度，将正则化系数的超参数 λ 设置为与迭代次数相关的动态值，其数学表达式为：

$$\lambda = \frac{2}{(-10i - 1)^{1+e^n}} \quad (38)$$

式中， i 为迭代次数； n 为预先设置的总迭代次数。

表 6-1 模型训练过程

算法
输入: 给定有标签的源域$X_S = \{x_i\}_{i=1}^n, Y_s = \{y_i\}_{i=1}^n$和一个无标签目标域$X_T = \{x_j\}_{j=1}^m$;经过任务一所述的特征提取算法, 提取特征。 输出: 训练完成的迁移学习网络; 目标域样本分类结果。 1: 开始 2: 使用任务二中的最佳权重初始化 ViT 模型。 3: 循环 4: 根据式 (35) 计算 MMD。 5: 根据式 (36) 计算 CORAL 损失。 6: 计算源域样本的交叉熵损失。 7: 根据式 (39) 计算目标函数。 8: 循环结束 (达到预设迭代次数) 9: 在其他目标域样本上测试。

模型训练过程中损失函数变化如下:

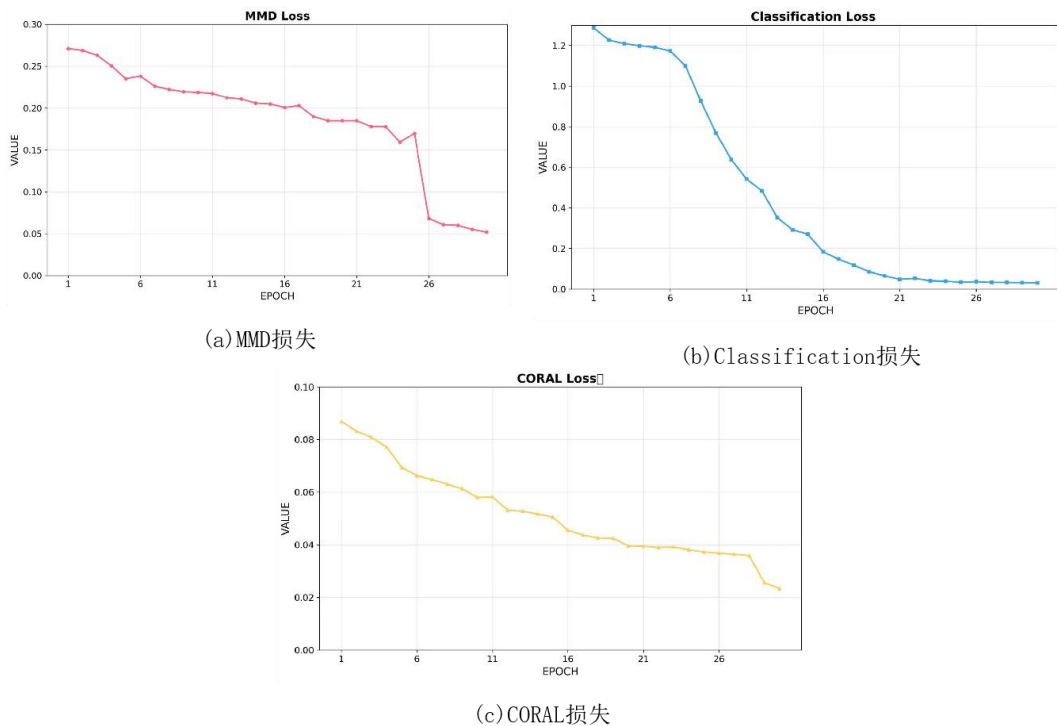


图 6.4 损失可视化

如图 6.4(a) - (c)所示, 三类损失在训练过程中表现出一致的收敛趋势。分类损失由约 1.3 持续下降并在 15 - 20 个 epoch 后稳定在约 0.03, 表明模型在有标签数据上已充分收敛且判别性逐步增强; CORAL 损失从约 0.085 平滑降至约 0.025, 说明源域与目标域特征的二阶统计 (协方差) 不断靠拢、域间偏移被有效削弱; MMD 损失整体下降, 并在约第 26 个 epoch 出现明显跃迁 (约 0.17 降至 0.05), 提示该阶段分布对齐显著加强, 通常与学习率或对齐损失权重的调度、批次构成变化等训练策略相关。综合来看, 模型在保持良好判

别性的同时，域不变表征持续增强，训练过程总体稳定

6.4 结果可视化分析

表 6-2 展示了将问题三在源域数据集上训练的模型迁移到目标域数据集上进行故障诊断的结果。图 6.5 清晰的展示了各类故障分布情况。

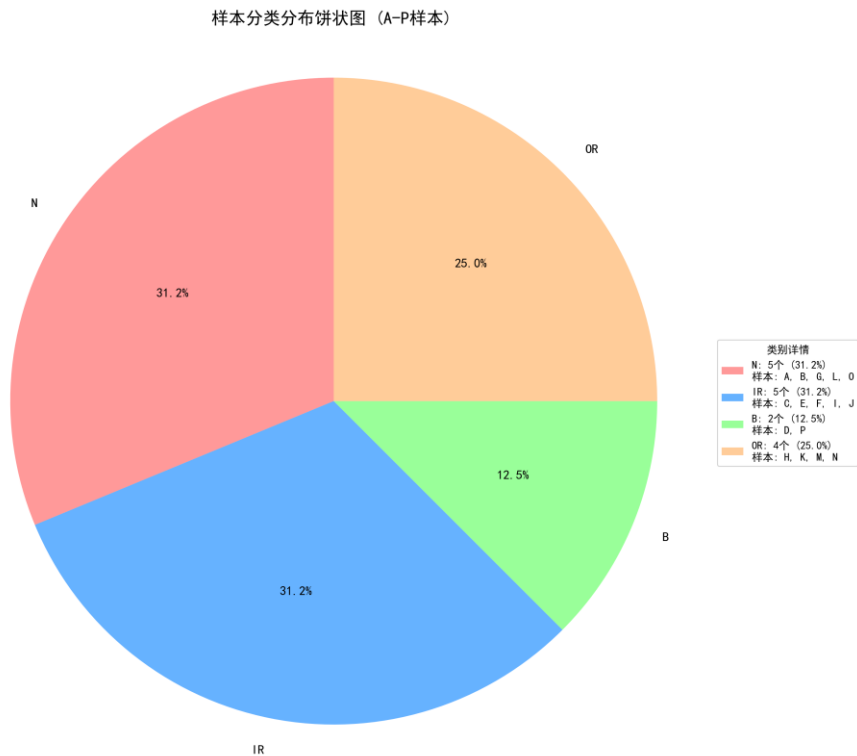


图 6.5 目标域样本分类分布图

从整体来看，模型能够对 16 个目标样本进行准确分类，预测类别覆盖正常状态（N）、内圈故障（IR）、外圈故障（OR）和滚动体故障（B）四类工况。模型诊断结果的平均置信度达到 0.992，多数样本的预测概率接近 0.900，这表明模型在迁移后仍保持了高度稳定的判别能力。值得注意的是，个别样本（如 O 文件，置信度 0.942；D 文件，置信度 0.968）置信度相对较低，表明在该类样本中可能存在工况复杂性或信号干扰对模型判决带来的不确定性。

总体而言，迁移学习有效缓解了源域与目标域之间的数据分布差异，使模型能够在目标域保持较高的分类可信度和良好的泛化性能，为高速列车轴承的工程化智能诊断提供了可行的技术路径。

表 6-2 模型迁移后故障诊断结果

数据文件名	故障类型	置信度
A	N	0.796
B	N	0.699
C	IR	0.999
D	B	0.968

E	IR	0.998
F	IR	0.997
G	N	0.892
H	OR	0.998
I	IR	0.999
J	IR	0.868
K	OR	0.879
L	N	0.897
M	OR	0.867
N	OR	0.999
O	N	0.942
P	B	0.997

7 问题 4：基于迁移学习的轴承故障诊断可解释性分析

7.1 问题分析

可解释性是机器学习的重要研究方向。受制于模型的“黑箱”属性，其迁移与诊断过程往往难以观测与理解，易引发对结果的不信任或盲目信任，进而制约诊断模型的工程应用。为此，本研究旨在提升迁移诊断的透明性与可验证性，避免模型在跨工况/跨设备场景中沦为“黑箱”。本文从以下三个方面展开分析：

（1）从散乱到对齐的跨域特征演化：通过特征空间的可视化与度量，展示迁移前后的分布变化与对齐效果。

（2）质心—方差对齐与距离分布：量化域级统计对齐程度及残余偏差，评估对跨域鲁棒性的贡献。

（3）事后可解释性分析：结合注意力可视化，审计单次预测的证据与可信度，验证其与轴承故障机理的一致性。

具体流程如图 7.1 所示。

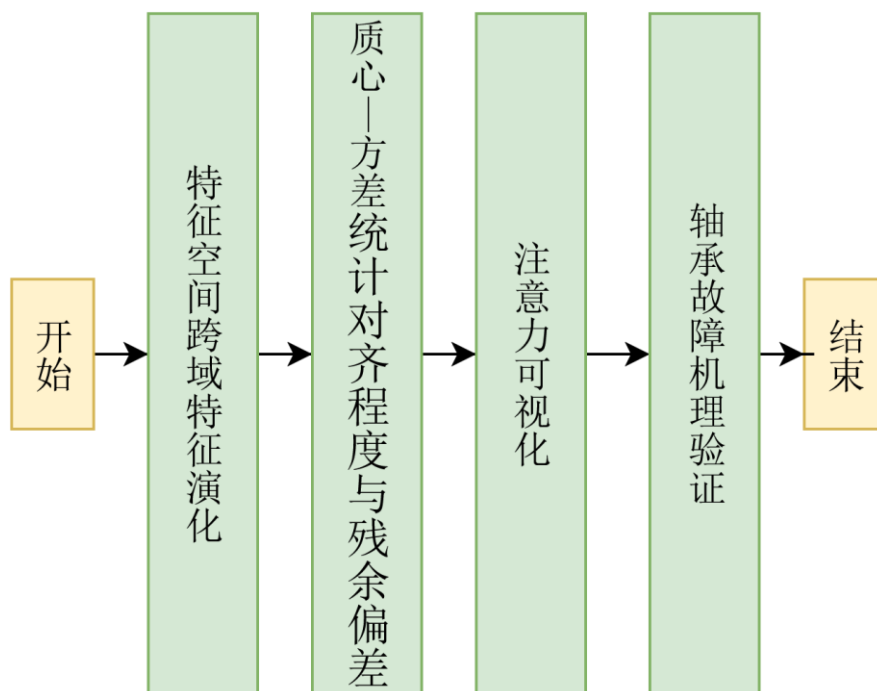


图 7.1 问题四流程图

7.2 迁移过程可解释性分析

7.2.1 从散乱到对齐的跨域特征演化

通过对特征空间动态变化的可视化，可以直观评估迁移学习的成效。在域适配场景中，t-SNE 投影是一种有效的诊断工具。图 7.2 (a) 展示了使用任务二中预训练的 ViT 直接提取特征后的结果：目标域样本（红色星形标记，T1-T16）在嵌入空间中高度分散且近乎随机，难以与源域样本（灰色点）形成稳定对应，明显反映了域偏移。就轴承故障机理而言，这种散乱主要源于工况变化（转速、载荷）和设备差异（安装、传感器位置、共振带）导致的特征频率缩放与能量分布偏移：例如内圈/外圈/滚动体/保持架对应的特征频率（BPFI、BPFO、BSF、FTF）及其倍频与侧频带在目标域位置和强度均发生改变，使得源域学到的模式无法在目标域上对齐。

图 7.2 (b) 给出了任务三中采用无监督领域自适应网络后目标域样本的显著变化。经过迁移学习（高阶与二阶联合矩匹配），目标域样本由“无序分散”转为“有序聚类”，并与源域样本形成清晰的一一对应关系（呈对角线式的类间分离格局）。二阶矩对齐促使模型匹配均值/协方差级别的能量分布与共振带结构，高阶矩对齐则强化对冲击稀疏性与包络峭度等非高斯特征的敏感性。结果是，模型在目标域学到的特征不再依赖设备/工况特有的局部噪声模式，而是聚焦于与故障机理一致的结构线索：包络谱中 BPFI/BPFO/BSF/FTF 及其倍频、由转速调制产生的侧频带、以及位于结构共振区间的解调能量抬升。不同故障类型样本各自聚为独立簇，说明模型提取到了具有阶次一致性与调制纹理稳定性的判别特征，从而实现跨域对齐。

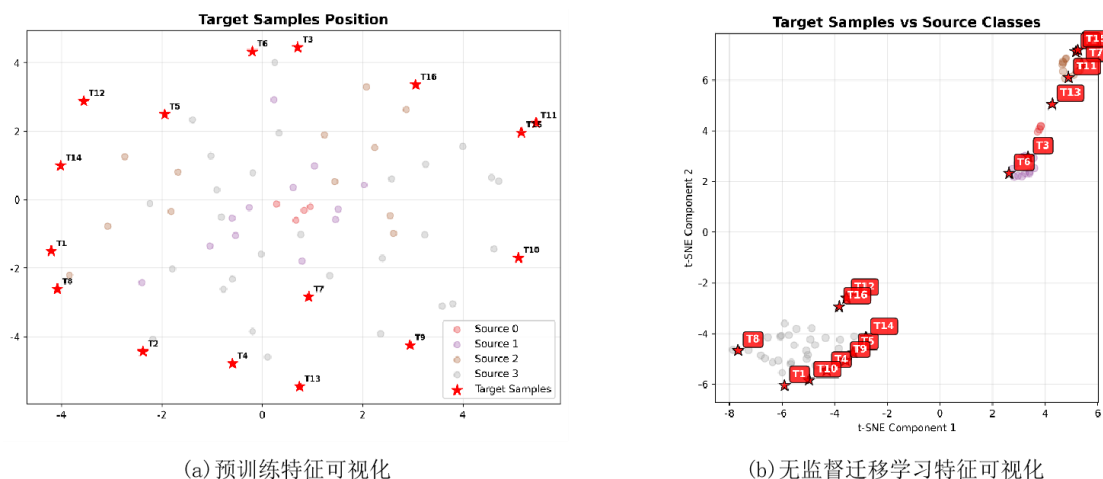


图 7.2 故障特征可视化

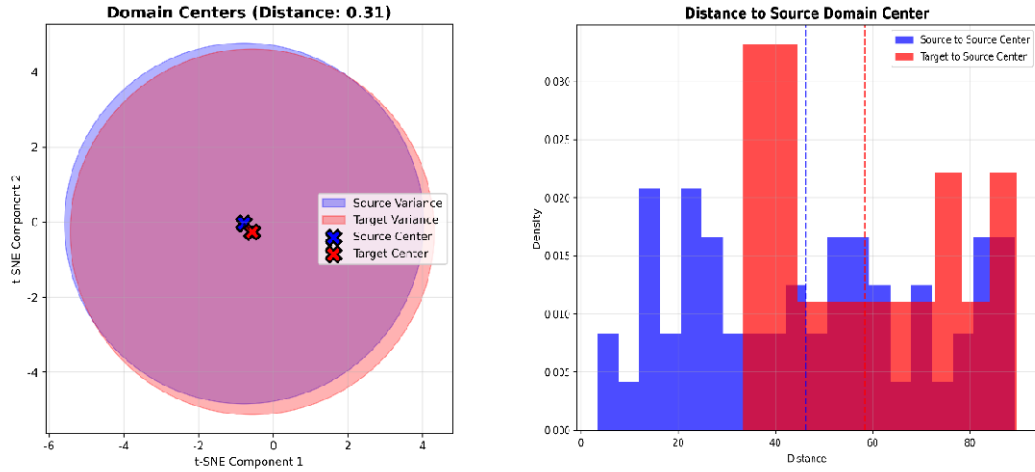
这一分布的跃迁表明模型在新的工况下仍能保持高精度识别：一方面，类内紧致、类间分离的聚类形态显示模型保留了广度与深度兼备的类别区分能力；另一方面，聚类与源域对齐体现了特征与物理机理的同构性，增强了迁移过程的可解释性与可信度。对工业故障诊断而言，这不仅提升了模型在无标签目标域的泛化能力和对不同工况的故障感知能力，也为迁移技术在工程应用中提供了坚实的机理支撑与实践依据。

7.2.2 质心—方差对齐与距离分布

图 7.3(a)展示了基于任务三的 Vision Transformer 结合高阶与二阶联合矩匹配的无监督领域自适应后源/目标域在 t-SNE 空间中的质心与方差范围。两域质心的欧氏距离仅为 0.31，且方差覆盖区域高度重叠，说明模型在均值与协方差层面实现了充分对齐，域间全局偏移被显著削弱。与轴承故障机理一致，这表明网络已将注意力聚焦到跨工况稳定的结构性特征上，包括包络谱中的 BPF1/BPFO/BSF/FTF 及其倍频与侧频带，以及位于结构共振区间的解调能量增强。

作为对照，图 7.3(b)为未进行迁移学习时的结果，并统计了样本到“源域中心”的距离分布。可以看到，目标域样本的分布相较源域整体明显右移，均值更大且尾部更厚，这表明存在显著的类内散度增大与高阶形状差异。其成因可能包括转速/载荷变化引起的特征频率缩放、传感器安装与结构共振差异导致的能量重分配，以及不同故障严重程度与类别占比差异造成的类条件分布偏移。

综合两幅图对比可得：所采用的无监督领域自适应（高阶与二阶联合矩匹配）在域级对齐上取得了实质性进展，使源/目标域的特征中心与总体离散度基本一致，从而为跨域识别奠定了稳健的公共子空间。同时，距离分布仍呈一定程度的右移与厚尾，揭示了残余的类条件差异与高阶统计偏差，提示后续可结合类原型/簇级对齐、基于阶次归一化的特征标准化与对比学习等策略，进一步压缩目标域的类内散度，提升复杂工况下的故障识别鲁棒性。



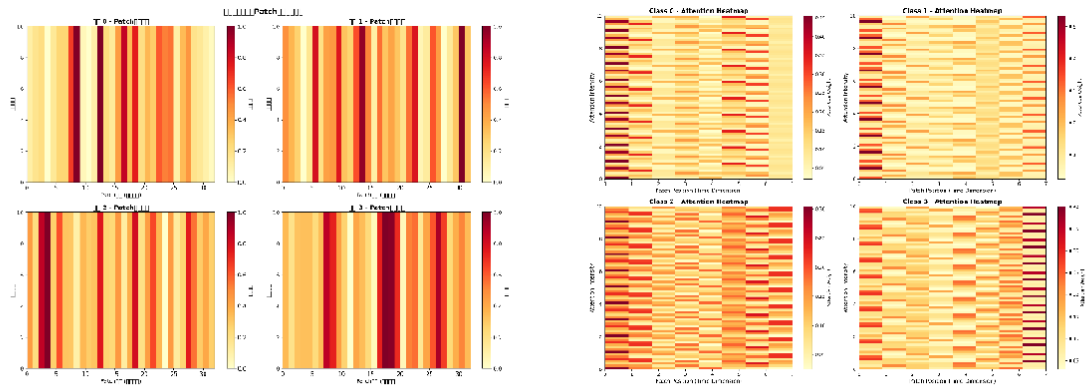
(a) 无监督迁移学习的域间距离

(b) 迁移学习之前的域间距离

图 7.3 域间距离对比图

7.3 事后可解释性分析

在事后可解释性中，注意力机制的可视化不仅能展示模型“看到了什么”，更可将关注区域与轴承故障的物理机理一一对应，从而提升诊断结论的可信度。图 7.4 给出了任务二（源域）与任务三（目标域）最后一层的注意力权重分布。对比可见，模型在迁移前后对信号特征的关注模式发生了显著变化，这种变化与典型轴承故障机理及其可观测特征相吻合。在任务二中，预训练（源域）模型的注意力呈尖峰状，主要集中于少数固定区域，往往对应于个别时间片的冲击脉冲或窄频带的共振响应。这意味着模型更依赖局部、低维特征（如某次冲击或单一频带能量峰），对全局调制结构关注不足。以机理解释，这些尖峰可能捕捉到单次或少量周期性冲击（由滚动体跨越缺陷产生），但对包络谱中更完整的故障特征频率与侧频结构覆盖有限：例如内圈故障的 BPFI、外圈故障的 BPFO、滚动体故障的 BSF 以及保持架故障的 FTF 及其倍频、转速侧频带等。此时，源域模型容易“记住”与传感器安装、结构共振相关的局部模式，而对跨工况（转速/负载变化导致特征频率缩放）缺乏鲁棒性。



(a) 预训练ViT的注意力矩阵

(b) 无监督领域自适应注意力矩阵

图 7.4 注意力矩阵对比

迁移到任务三后（采用高阶与二阶联合矩匹配），目标域模型的注意力分布明显更平滑、更均衡，并覆盖更广的时频区域。二阶矩对齐（均值/协方差）帮助模型适应能量谱与共振带的整体偏移，高阶矩对齐（偏度/峭度）强化了对冲击与稀疏突变的敏感性，因而模型不再仅依赖单点峰值，而是同时关注与故障机理相关的一组结构性线索：包络谱中

BPFI/BPFO/BSF/FTF 及其倍频、随转速产生的侧频带，以及位于结构共振区间的解调能量抬升。这种注意力的“扩展与均衡”体现了模型对目标域更稳定的物理一致性建模——能够在阶次归一化后仍聚焦相同的故障阶次峰值，并在时频图上捕获由调制产生的条带与旁瓣，从而提升对复杂故障模式的感知与迁移适应能力。

从类别层面看，注意力图同样体现出与故障机理一致的差异性：外圈故障通常在固定空间位置激励结构共振，包络谱在 BPFO 及其倍频处峰值突出，注意力更集中于相对稳定的频带；内圈故障随旋转部件运动，侧频更明显、倍频更丰富，注意力在多阶次与侧频带间分布更分散；滚动体故障常伴随非线性接触与调制效应，注意力覆盖更宽的频带与对角时频纹理；保持架故障则在低频 FTF 及其倍频附近有显著关注。这些差异表明模型在完成领域自适应的同时，仍保留并强化了不同故障类别的物理显著性，降低了因特征过度泛化而引发的类别混淆风险。

8 模型的优缺点分析

8.1 模型的优点

(1)数据预处理全面：在问题一中，先通过采样率统一、TVFEMD 与 GWO-MOMEDA 的组合降噪以及 SMOTE 样本平衡处理，最大限度提升了信号的纯净度和数据均衡性，为后续建模提供了可靠的数据基础。

(2)多域特征融合：构建了时域、频域与熵域的 36 项特征指标，并借助 MixHop 进行多域特征融合，既保证了特征多样性，又增强了跨域特征的表达能力。

(3)领域自适应策略先进：在问题三中，引入二阶与高阶统计量联合匹配，并结合 ViT 构建无监督领域自适应网络，有效缓解了源域与目标域之间的分布偏移问题，提升了跨域诊断的可迁移性。

8.2 模型的缺点

(1)对极端工况的适应性有限：尽管进行了分布对齐，但在实际复杂运行环境下，噪声干扰和样本极度稀缺仍可能导致模型性能下降。

(2)参数依赖性强：TVFEMD 分解参数、MOMEDA 滤波参数以及 MixHop 的层数和邻阶设定均可能显著影响结果，需通过实验精调，增加了模型调参难度。

9 参考文献

- [1] 邵鲁川,赵兵,康旭涛.基于 DE 优化 VMD 与 CNN-BiGRU-Attention 结合的滚动轴承故障诊断[J/OL].电子测量技术,1-13[2025-09-23].
- [2] 寇梓良,张西宁,李兵,等.基于多模态数据融合与改进 CNN 的轴承故障诊断[J/OL].振动、测试与诊断,1-8[2025-09-23].
- [3] 丛晓妍,王增才,王保平,等.基于 EMD 与峭度滤波的煤岩界面识别[J].振动.测试与诊断,2015,35(05):950-954.
- [4] 宋硕.基于自适应时变经验模态分解及多尺度熵特征融合的轴承故障诊断研究[D].武汉大学,2024.
- [5] Li H ,Li Z ,Mo W .A time varying filter approach for empirical mode decomposition[J].Signal Processing,2017,138146-158.
- [6] YeXR,HuYF, Shen JX, et al.An adaptive optimizedTVF-EMDbased on a sparsity-impact measure index for bearing incipient fault diagnosis[J]. IEEE TransactiononInstrumentationandMeasurement,2021,70,1-11.
- [7] 赵雅雪,王旭,蒋传文等.基于最大信息系数相关性分析和改进多层级门控 LSTM 的短期电价预测方法[J].中国电机工程学报,2021,41(01):135-146+404.
- [8] 李欣海.随机森林模型在分类与回归分析中的应用[J].应用昆虫学报,2013,50(04):1190-1197.
- [9] 曾钰.基于信号处理与深度学习的铁路列车走行部滚动轴承智能故障诊断方法研究[D].北京交通大学,2022.
- [10] Abu-El-Haija S ,Perozzi B ,Kapoor A , et al.MixHop: Higher-Order Graph Convolutional Architectures via Sparsified Neighborhood Mixing.[J].CoRR,2019,abs/1905.00067
- [11] 袁新杰,孙飞越.基于短时傅立叶变换和改进 Vision Transformer 的滚动轴承故障诊断方法[J].起重运输机械,2024,(16):70-75.

10 附录

```
1. function show_save_plot(x,path)
2. subplot(2,1,1);
3. data = x;
4. plot(data,"Color","#32B897");
5. title("(a) 轴承内圈故障信号","FontWeight","bold","FontSize",10,"FontName","宋体");
6. xlabel("采样点");
7. ylabel("幅值");
8. set(gca, 'Color', [0.99 0.99 0.99]);
9. ret = envelope(data,162,"analytic");
10. subplot(2,1,2);
11. % ret(1:1000:200)
12. plot(ret(1:1000),"Color","#32B897");
13. title("(b) 轴承内圈故障包络谱","FontWeight","bold","FontSize",10,"FontName","宋体");
14. xlabel("频率(Hz)");
15. ylabel("幅值");
16. saveas(gcf,path);
17. end

18. clc,clear;
19. path_x = "D:\E 题数据集\源域数据集\48kHz_DE_data\IR\0007/";
20. path_y = "IR007_0";
21. path = path_x+path_y+".mat";
22. mat = load(path);
23. [z,x,y] = whos('-file',path).name;
24. show_save_plot(mat.(x),"D:\fig\48kHz\IR\"+path_y+"_"+x+".png");
25. show_save_plot(mat.(y),"D:\fig\48kHz\IR\"+path_y+"_"+y+".png");
```

绘制信号和包络谱 matlab 代码

```
1. from scipy.io import loadmat
2. import pandas as pd
3. import numpy as np
4. from rich.console import Console

5. def mat2table(path: str):
6.     c = Console()
7.     idx = 0
8.     with c.status("[blod red]正在转换") as status:
9.         for i in os.listdir(path):
10.             if i.endswith(".mat"):
11.                 mat_file_path = os.path.join(path, i)
12.                 print(mat_file_path)
13.                 mat_data = loadmat(mat_file_path)
```

```

14.         mat_data.pop("__header__")
15.         mat_data.pop("__version__")
16.         mat_data.pop("__globals__")
17.         for j in mat_data.keys():
18.             mat_data[j] = mat_data[j].squeeze()
19.         df = pd.DataFrame.from_dict(mat_data, orient='columns')
20.         df.iloc[1:, 3] = np.nan
21.         # print(df.head())
22.         df.to_excel(os.path.join(path, i.split(".")[0] + ".xlsx"), index=False)
23.         idx += 1
24.         status.update("[blod green] 第{}个处理完成".format(idx))
25.         # df = pd.DataFrame(mat_data)
26.         # print(df)

```

27. mat2table(r"D:\E\数据集\源域数据集\12kHz_DE_data\OR\Centered\0007")
将.mat 文件转为 excel 表格

```

1.  clc,clear,close all;
2.  data = load("D:\E 题数据集\源域数据集\48kHz_DE_data\B\0007\B007_0.mat");
3.  imfs = tvf_emd(data.X122_DE_time);
4.  [n_imf, n_sample] = size(imfs);
5.  fprintf('检测到 IMF 分量: %d 个, 每个分量包含%d 个采样点\n', n_imf, n_sample);

6.  % 步骤 1: 计算每个 IMF 分量的原始峭度值
7.  kurtosis_values = zeros(1, n_imf);
8.  for i = 1:n_imf
9.      imf_component = imfs(i, :); % 提取第 i 个 IMF 分量 (行向量)
10.     kurtosis_values(i) = kurtosis(imf_component); % 计算峭度
11. end

12. % 步骤 2: 计算每个 IMF 分量的权重 (基于能量)
13. energy = zeros(1, n_imf);
14. for i = 1:n_imf
15.     imf_component = imfs(i, :);
16.     energy(i) = sum(imf_component.^2); % 单个分量的能量 (平方和)
17. end
18. total_energy = sum(energy);
19. weights = energy / total_energy; % 归一化权重 (总和=1)

20. % 步骤 3: 计算每个 IMF 分量的加权峭度值 (峭度×权重)
21. imf_weighted_kurtosis = kurtosis_values .* weights;

22. % 步骤 4: 计算总加权峭度 (所有分量加权峭度的总和, 可选)
23. total_weighted_kurtosis = sum(imf_weighted_kurtosis);

```

```

24. % 显示结果
25. fprintf('\n===== 各 IMF 分量的加权峭度值 =====\n');
26. result_table = table((1:n_imf)', kurtosis_values', weights', imf_weighted_kurtosis', ...
27.     'VariableNames', {'IMF 索引', '原始峭度值', '权重', '加权峭度值'});
28. disp(result_table);
29. fprintf('\n所有分量的总加权峭度值: %.4f\n', total_weighted_kurtosis);

30. figure;
31. subplot(2,2,1);
32. mean_imf_weighted_kurtosis = mean(imf_weighted_kurtosis);
33. plot(1:10,imf_weighted_kurtosis,'--*r',...
34.     'LineWidth',1,...
35.     'MarkerSize',5,...
36.     'MarkerEdgeColor','r','Color','b');
37. hold on ;
38. plot(0:n_imf,mean_imf_weighted_kurtosis*ones([1,n_imf+1]),"Color","r","LineWidth",1.0,"LineStyle","--");
39. text(n_imf*0.8,mean_imf_weighted_kurtosis+.1,sprintf('Mean: %.2f%%', mean_imf_weighted_kurtosis), 'Color', [74/255, 74/255, 74/255], ... % 深灰色
40.     'FontSize', 9);
41. xlabel("IMF 序号", 'FontSize', 10, 'FontWeight', 'bold');
42. ylabel("WESK 值", 'FontSize', 10, 'FontWeight', 'bold');
43. title("(a)各 IMF 分量的加权谱峭度值", "FontWeight", "bold", "FontSize", 10, "FontName", "宋体");
44. hold off;
45. % 画第二幅图
46. subplot(2,2,2);
47. selected_indices = [1, 5, 8, 9]; % 目标 IMF 索引 (IMF1、IMF5、IMF8、IMF9)
48. reconstructed_signal = sum(imfs(selected_indices, :)); % 重构信号 = 选中 IMF 之和
49. plot(reconstructed_signal);
50. xlabel("采样点", 'FontSize', 10, 'FontWeight', 'bold')
51. ylabel("幅度", 'FontSize', 10, 'FontWeight', 'bold')
52. title("(b)筛选后 IMF 分量的重构信号", "FontWeight", "bold", "FontSize", 10, "FontName", "宋体");
53. % 画第三幅图
54. subplot(2,2,[3,4]);
55. bl_g = envelope(reconstructed_signal(1:1000),141,"analytic");
56. plot(bl_g);
57. xlabel("频率(Hz)", 'FontSize', 10, 'FontWeight', 'bold')
58. ylabel("幅值", 'FontSize', 10, 'FontWeight', 'bold')
59. title("(c)TVFEMD 重构信号的包络图", "FontWeight", "bold", "FontSize", 10, "FontName", "宋体");

```

数据处理绘图的 matlab 代码

```

1. import polars as pl
2. import numpy as np
3. import pandas as pd
4. from PyEMD import EMD

```

```

5. from sklearn.model_selection import train_test_split

6. df = pl.read_csv(r"D:\E 题数据集\data_48k_de_52c_vmd_denoised(1).csv")
7. df = df.drop("type").to_numpy()

8. i, j = df.shape
9. result = []
10. for i in range(i):
11.     label = df[i, -1]
12.     x = df[i, :j - 1:]
13.     for k in range(0, j, 1024):
14.         tmp = df[i, k:k + 2048]
15.         if len(tmp) == 2048:
16.             result.append(np.append(tmp, label))
17.     # break
18. data = np.array(result)
19. data_i, data_j = data.shape

20. emd_obj = EMD()
21. emd_result = []
22. for i in range(data_i):
23.     emd_imfs = emd_obj.emd(data[i, :-1:], max_imf=5)
24.     if emd_imfs.shape[0] == 2:
25.         continue
26.     res = np.concat([emd_imfs[:5:, :], np.ones((emd_imfs.shape[0] - 1, 1)) * data[i, -1]],
27.                     axis=1)
28.     emd_result.append(res)
29. emd_result_arr = np.array(emd_result)
30. emd_result_arr = emd_result_arr.sum(axis=1)
31. emd_result_arr[:, -1] /= 5

```

Emd 处理数据代码，（emd 拼接操作）

```

1. import numpy as np
2. import scipy.stats as stats
3. from scipy.fftpack import fft
4. from scipy.signal import hilbert, welch
5. import pywt
6. import antropy as ant
7. from PyEMD import EMD

8. # ===== 输入: 时间序列 =====
9. def approximate_entropy(U, m, r):
10.     N = len(U)
11.     def _phi(m):
12.         X = np.array([U[i:i + m] for i in range(N - m + 1)])

```

```

13.         C = []
14.         for x in X:
15.             dist = np.max(np.abs(X - x), axis=1)
16.             C.append(np.sum(dist <= r) / (N - m + 1.0))
17.         return np.sum(np.log(C)) / (N - m + 1.0)

18.     return abs(_phi(m) - _phi(m + 1))

19. def feature_extraction(x, fs=1000):
20.     x = x.flatten()
21.     N = len(x)
22.     # ----- 时域特征 -----
23.     mean_val = np.mean(x)
24.     var_val = np.var(x)
25.     rms_val = np.sqrt(np.mean(x ** 2)) # 方根幅值
26.     abs_mean = np.mean(np.abs(x)) # 绝对平均幅值
27.     max_val = np.max(x)
28.     min_val = np.min(x)
29.     peak_val = max(abs(max_val), abs(min_val)) # 峰值
30.     effective_val = rms_val
31.     skewness = stats.skew(x)
32.     kurtosis = stats.kurtosis(x)
33.     # 指标类
34.     crest_factor = peak_val / rms_val
35.     shape_factor = rms_val / abs_mean
36.     impulse_factor = peak_val / abs_mean
37.     margin_factor = peak_val / (np.mean(np.sqrt(np.abs(x))) ** 2)
38.     skewness_index = skewness
39.     kurtosis_index = kurtosis
40.     # ----- 频域特征 -----
41.     freqs = np.fft.fftfreq(N, 1 / fs)
42.     X = np.abs(np.fft(x))[:N // 2]
43.     freqs = freqs[:N // 2]
44.     amp_mean = np.mean(X)
45.     spectral_centroid = np.sum(freqs * X) / np.sum(X)
46.     mean_square_freq = np.sum((freqs ** 2) * X) / np.sum(X)
47.     var_freq = mean_square_freq - spectral_centroid ** 2
48.     rms_freq = np.sqrt(mean_square_freq)
49.     amp_var = np.var(X)
50.     amp_abs_mean = np.mean(np.abs(X))
51.     amp_skewness = stats.skew(X)
52.     amp_kurtosis = stats.kurtosis(X)
53.     std_freq = np.sqrt(var_freq)
54.     # 频域频率分布

```

```

55.     prob_freq = X / np.sum(X)
56.     freq_skewness = np.sum(((freqs - spectral_centroid) ** 3) * prob_freq) / (std_freq **
3)
57.     freq_kurtosis = np.sum(((freqs - spectral_centroid) ** 4) * prob_freq) / (std_freq **
4)
58.     # ----- 比率指标 -----
59.     square_root_amplitude = np.sqrt(np.mean(np.sqrt(np.abs(x))) ** 2)
60.     square_root_ratio = rms_val / square_root_amplitude
61.     # ----- 熵类 -----
62.     apen = ant.app_entropy(x, order=2, metric='chebyshev')
63.     # 样本熵
64.     sampen = ant.sample_entropy(x, order=2, metric='chebyshev')
65.     # 包络熵
66.     analytic_signal = hilbert(x)
67.     envelope = np.abs(analytic_signal)
68.     prob_env, _ = np.histogram(envelope, bins=50, density=True)
69.     prob_env = prob_env[prob_env > 0]
70.     env_entropy = -np.sum(prob_env * np.log(prob_env))
71.     # 功率谱熵
72.     f, Pxx = welch(x, fs=fs, nperseg=256)
73.     Pxx_norm = Pxx / np.sum(Pxx)
74.     ps_entropy = -np.sum(Pxx_norm * np.log(Pxx_norm))
75.     # EMD 能量熵
76.     emd = EMD()
77.     imfs = emd(x)
78.     energies = [np.sum(imf ** 2) for imf in imfs]
79.     energies = np.array(energies) / np.sum(energies)
80.     emd_entropy = -np.sum(energies * np.log(energies + 1e-12))
81.     # 小波包能量熵
82.     wp = pywt.WaveletPacket(data=x, wavelet='db4', mode='symmetric', maxlevel=3)
83.     nodes = wp.get_level(3, order="freq")
84.     wp_energies = np.array([np.sum(np.square(node.data)) for node in nodes])
85.     wp_energies /= np.sum(wp_energies)
86.     wp_entropy = -np.sum(wp_energies * np.log(wp_energies + 1e-12))
87.     fuzzy_entropy = sampen
88.     return {
89.         "mean": mean_val, "variance": var_val, "RMS": rms_val, "abs_mean": abs_mean,
90.         "max": max_val, "min": min_val, "peak": peak_val, "effective": effective_val,
91.         "skewness": skewness, "kurtosis": kurtosis, "crest_factor": crest_factor,
92.         "shape_factor": shape_factor, "impulse_factor": impulse_factor,
93.         "margin_factor": margin_factor, "skewness_index": skewness_index,
94.         "kurtosis_index": kurtosis_index, "amp_mean": amp_mean,
95.         "spectral_centroid": spectral_centroid, "mean_square_freq": mean_square_freq,
96.         "var_freq": var_freq, "rms_freq": rms_freq, "amp_var": amp_var,

```

```

97.         "amp_abs_mean": amp_abs_mean, "amp_skewness": amp_skewness,
98.         "amp_kurtosis": amp_kurtosis, "std_freq": std_freq, "freq_skewness": freq_skewness,
99.         "freq_kurtosis": freq_kurtosis, "square_root_ratio": square_root_ratio,
100.        "ApEn": apen, "SampEn": sampen, "envelope_entropy": env_entropy,
101.        "ps_entropy": ps_entropy, "EMD_entropy": emd_entropy, "WP_entropy": wp_entropy,
102.        "fuzz_y_entropy": fuzzy_entropy}
103. def feature_get(emd_result_arr, label_flag: bool = False):
104.     feature_list = []
105.     c = len(emd_result_arr)
106.     if label_flag:
107.         for i in range(c):
108.             feature = feature_extraction(emd_result_arr[i][: -1:], fs=48000)
109.             feature["label"] = emd_result_arr[i][ -1]
110.             feature_list.append(feature)
111.     else:
112.         for i in range(c):
113.             feature = feature_extraction(emd_result_arr[i][::], fs=48000)
114.             feature_list.append(feature)
115.     return feature_list
116. fg_df = feature_get(targe_df.to_numpy())

```

提取 36 特征

```

1. import seaborn as sns
2. import matplotlib.pyplot as plt
3. from matplotlib import gridspec

4. fig, axs = plt.subplots(6, 3, figsize=(20, 20))
5. # 调整子图之间的间隔
6. plt.subplots_adjust(hspace=0.5, wspace=0.3) # 控制高度和宽度方向的间隔
7. idx = 18
8. for i in range(6):
9.     for j in range(3):
10.        plt.subplot(6, 3, idx - 18 + 1)
11.        sns.kdeplot(data=target, x=f'{target.columns[idx]}', fill=True, legend=True)
12.        plt.title("KDE of {}".format(target.columns[idx]))
13.        plt.xlabel("{}".format(target.columns[idx]))
14.        plt.ylabel("Density")
15.        sns.kdeplot(data=source, x=f'{source.columns[idx]}', fill=True, legend=True)
16.        plt.title("KDE of {}".format(source.columns[idx]))
17.        plt.xlabel("{}".format(source.columns[idx]))
18.        plt.ylabel("Density")
19.        idx += 1
20.        plt.legend(labels=["target domain", "source domain"])
21. plt.savefig("D://kde.png")

```

绘制 KDE 图

```

1. import numpy as np
2. import matplotlib.pyplot as plt
3. from sklearn.preprocessing import StandardScaler

4. def process_and_visualize_features():
5.     plt.subplots(2, 2, figsize=(20, 20))
6.     plt.subplots_adjust(hspace=0.5, wspace=0.3)
7.     original_data = source.to_numpy()
8.     data_subset = [original_data[:40, :], original_data[40:80, :],
9.     original_data[80:120, :], original_data[120:140, :]] # 1, 2, 3, 0
10.    d = ["(a)", "(c)", "(b)", "(d)"]
11.    for i in range(4):
12.        # 步骤 3: 对选取的40 行数据进行标准化处理
13.        # 标准化:  $(X - \text{mean}) / \text{std\_dev}$ 
14.        plt.subplot(2, 2, i + 1)
15.        scaler = StandardScaler()
16.        scaled_data = scaler.fit_transform(data_subset[i])
17.        c = plt.pcolormesh(scaled_data, cmap="rainbow")
18.        plt.colorbar(c, orientation='horizontal', pad=0.05)
19.        plt.title(f"{d[i]}", fontsize=16, y=-0.2)
20.        plt.tight_layout()
21.    plt.savefig("./heatmap.png")
22.    if __name__ == '__main__':
23.        plt.rcParams['font.sans-serif'] = ['SimHei']
24.        plt.rcParams['axes.unicode_minus'] = False
25.    process_and_visualize_features()

```

绘制特征图

问题 2:

```

1. import pandas as pd
2. import numpy as np
3. import torch
4. import torch.nn as nn
5. from torch.utils.data import Dataset, DataLoader
6. from sklearn.preprocessing import StandardScaler, LabelEncoder
7. from sklearn.model_selection import train_test_split
8. import matplotlib.pyplot as plt
9. from tqdm import tqdm
10. import warnings
11. warnings.filterwarnings('ignore')

12. # 设置随机种子
13. torch.manual_seed(42)
14. np.random.seed(42)
15. class WindowedDataset(Dataset):

```

```

16.         def __init__(self, data, labels, window_size=2048, step_size=1024):
17.             self.data = data
18.             self.labels = labels
19.             self.window_size = window_size
20.             self.step_size = step_size
21.             # 生成所有窗口
22.             self.windows = []
23.             self.window_labels = []
24.             for i, (sequence, label) in enumerate(tqdm(zip(data, labels), total=len(data),
desc="生成窗口")):
25.                 # 对每个序列进行滑动窗口
26.                 sequence = np.array(sequence)
27.                 num_windows = 0
28.                 for start in range(0, len(sequence) - window_size + 1, step_size):
29.                     end = start + window_size
30.                     window = sequence[start:end]
31.                     self.windows.append(window)
32.                     self.window_labels.append(label)
33.                     num_windows += 1
34.                 self.windows = np.array(self.windows)
35.                 self.window_labels = np.array(self.window_labels)
36.             def __len__(self):
37.                 return len(self.windows)
38.             def __getitem__(self, idx):
39.                 return torch.FloatTensor(self.windows[idx]), torch.LongTensor([self.window_labels[idx]])[0]
40.         class ViT1D(nn.Module):
41.             def __init__(self, seq_len=2048, patch_size=64, num_classes=2, dim=256, depth=6, heads=8, mlp_dim=512):
42.                 super().__init__()
43.                 assert seq_len % patch_size == 0, f"序列长度 {seq_len} 必须能被 patch_size {patch_size} 整除"
44.                 self.patch_size = patch_size
45.                 self.num_patches = seq_len // patch_size
46.                 self.patch_dim = patch_size
47.                 # Patch embedding
48.                 self.patch_embedding = nn.Linear(self.patch_dim, dim)
49.                 # 位置编码
50.                 self.pos_embedding = nn.Parameter(torch.randn(1, self.num_patches + 1, dim))
51.
52.                 # CLS token
53.                 self.cls_token = nn.Parameter(torch.randn(1, 1, dim))
54.                 # Transformer blocks
55.                 self.transformer_blocks = nn.ModuleList([

```

```

56.         TransformerBlock(dim, heads, mlp_dim) for _ in range(depth)
57.     ])
58.     # 分类头
59.     self.classifier = nn.Sequential(
60.         nn.LayerNorm(dim),
61.         nn.Linear(dim, num_classes)
62.     )
63.     self.dropout = nn.Dropout(0.1)
64.     def forward(self, x):
65.         batch_size = x.shape[0]
66.         # 将序列分割成 patches
67.         x = x.view(batch_size, self.num_patches, self.patch_size)
68.         # Patch embedding
69.         x = self.patch_embedding(x)
70.         # 添加CLS token
71.         cls_tokens = self.cls_token.expand(batch_size, -1, -1)
72.         x = torch.cat([cls_tokens, x], dim=1)
73.         # 添加位置编码
74.         x += self.pos_embedding
75.         x = self.dropout(x)
76.         # Transformer blocks
77.         for transformer in self.transformer_blocks:
78.             x = transformer(x)
79.         # 使用CLS token 进行分类
80.         cls_output = x[:, 0]
81.         output = self.classifier(cls_output)
82.         return output

83.     class TransformerBlock(nn.Module):
84.         def __init__(self, dim, heads, mlp_dim):
85.             super().__init__()
86.             self.attention = MultiHeadAttention(dim, heads)
87.             self.norm1 = nn.LayerNorm(dim)
88.             self.norm2 = nn.LayerNorm(dim)
89.             self.mlp = nn.Sequential(
90.                 nn.Linear(dim, mlp_dim),
91.                 nn.GELU(),
92.                 nn.Dropout(0.1),
93.                 nn.Linear(mlp_dim, dim),
94.                 nn.Dropout(0.1)
95.             )
96.
97.         def forward(self, x):
98.             x = x + self.attention(self.norm1(x))

```

```

99.         x = x + self.mlp(self.norm2(x))
100.         return x
101.     class MultiHeadAttention(nn.Module):
102.         def __init__(self, dim, heads):
103.             super().__init__()
104.             self.heads = heads
105.             self.scale = (dim // heads) ** -0.5
106.             self.to_qkv = nn.Linear(dim, dim * 3, bias=False)
107.             self.to_out = nn.Sequential(
108.                 nn.Linear(dim, dim),
109.                 nn.Dropout(0.1)
110.             )
111.         def forward(self, x):
112.             b, n, d = x.shape
113.             qkv = self.to_qkv(x).chunk(3, dim=-1)
114.             q, k, v = map(lambda t: t.view(b, n, self.heads, d // self.heads).transpose(1,
115. 2), qkv)
116.             dots = torch.matmul(q, k.transpose(-1, -2)) * self.scale
117.             attn = dots.softmax(dim=-1)
118.             out = torch.matmul(attn, v)
119.             out = out.transpose(1, 2).contiguous().view(b, n, d)
120.             return self.to_out(out)
121.     def train_model(model, train_loader, val_loader, num_epochs=30, learning_rate=1e-4):
122.         device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
123.         model = model.to(device)
124.         criterion = nn.CrossEntropyLoss()
125.         optimizer = torch.optim.AdamW(model.parameters(), lr=learning_rate, weight_decay=1e-4)
126.         scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=num_epochs)
127.
128.         train_losses = []
129.         val_losses = []
130.         train_accs = []
131.         val_accs = []
132.         best_val_acc = 0
133.         for epoch in range(num_epochs):
134.             # 训练阶段
135.             model.train()
136.             train_loss = 0
137.             train_correct = 0
138.             train_total = 0
139.             pbar = tqdm(train_loader, desc=f"Epoch {epoch+1}/{num_epochs} - Training")
140.             for batch_idx, (data, target) in enumerate(pbar):

```

```

141.         data, target = data.to(device), target.to(device)
142.         optimizer.zero_grad()
143.         output = model(data)
144.         loss = criterion(output, target)
145.         loss.backward()
146.         optimizer.step()
147.         train_loss += loss.item()
148.         _, predicted = torch.max(output.data, 1)
149.         train_total += target.size(0)
150.         train_correct += (predicted == target).sum().item()
151.         # 更新进度条
152.         pbar.set_postfix({
153.             'Loss': f'{loss.item():.4f}',
154.             'Acc': f'{100.*train_correct/train_total:.2f}%'
155.         })
156.         # 验证阶段
157.         model.eval()
158.         val_loss = 0
159.         val_correct = 0
160.         val_total = 0
161.         with torch.no_grad():
162.             for data, target in tqdm(val_loader, desc="Validating"):
163.                 data, target = data.to(device), target.to(device)
164.                 output = model(data)
165.                 loss = criterion(output, target)
166.                 val_loss += loss.item()
167.                 _, predicted = torch.max(output.data, 1)
168.                 val_total += target.size(0)
169.                 val_correct += (predicted == target).sum().item()
170.         # 计算平均损失和准确率
171.         avg_train_loss = train_loss / len(train_loader)
172.         avg_val_loss = val_loss / len(val_loader)
173.         train_acc = 100 * train_correct / train_total
174.         val_acc = 100 * val_correct / val_total
175.         train_losses.append(avg_train_loss)
176.         val_losses.append(avg_val_loss)
177.         train_accs.append(train_acc)
178.         val_accs.append(val_acc)
179.         # 保存最佳模型
180.         if val_acc > best_val_acc:
181.             best_val_acc = val_acc
182.             torch.save(model.state_dict(), 'best_vit_model.pth')
183.         scheduler.step()
184.         return train_losses, val_losses, train_accs, val_accs

```

```

185.     def main():
186.         # 步骤1: 加载数据集
187.         df = pd.read_csv('data4_resampled_32khz_balanced.csv', header=None)
188.         # 分离特征和标签
189.         features = df.iloc[:, :-1].values
190.         labels = df.iloc[:, -1].values
191.         label_encoder = LabelEncoder()
192.         labels_encoded = label_encoder.fit_transform(labels)
193.         num_classes = len(np.unique(labels_encoded))
194.         # 数据标准化
195.         scaler = StandardScaler()
196.         features_scaled = scaler.fit_transform(features.T).T # 对每个时间点进行标准化
197.         # 划分训练集和验证集
198.         X_train, X_val, y_train, y_val = train_test_split(
199.             features_scaled, labels_encoded, test_size=0.2, random_state=42, stratify=labels_encoded
200.         )
201.         # 步骤2: 创建滑动窗口数据集
202.         train_dataset = WindowedDataset(X_train, y_train, window_size=2048, step_size=1024)
203.         val_dataset = WindowedDataset(X_val, y_val, window_size=2048, step_size=1024)
204.         # 创建数据加载器
205.         batch_size = 16 # 减小batch size 以适应内存
206.         train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True, num_workers=0)
207.         val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False, num_workers=0)
208.         # 步骤3: 创建ViT模型
209.         model = ViT1D(
210.             seq_len=2048,
211.             patch_size=64,
212.             num_classes=num_classes,
213.             dim=256,
214.             depth=6,
215.             heads=8,
216.             mlp_dim=512
217.         )
218.         total_params = sum(p.numel() for p in model.parameters())
219.         trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
220.         # 步骤4: 训练模型
221.         train_losses, val_losses, train_accs, val_accs = train_model(
222.             model, train_loader, val_loader, num_epochs=30, learning_rate=1e-4
223.         )
224.         return model, train_losses, val_losses, train_accs, val_accs
225.     if __name__ == "__main__":

```

```
226.         model, train_losses, val_losses, train_accs, val_accs = main()
```

问题三:

```
1.     import json
2.     import time
3.     import argparse
4.     import numpy as np
5.     import pandas as pd
6.     import matplotlib.pyplot as plt
7.     import torch
8.     import torch.nn as nn
9.     import torch.optim as optim
10.    import torch.nn.functional as F
11.    from torch.utils.data import Dataset, DataLoader, TensorDataset
12.    from sklearn.preprocessing import StandardScaler, LabelEncoder
13.    from sklearn.model_selection import train_test_split
14.    from sklearn.metrics import classification_report, accuracy_score
15.    import warnings
16.    warnings.filterwarnings('ignore')
17.    # 设置随机种子
18.    set_seed(42)
19.    class DomainDataset(Dataset):
20.        def __init__(self, source_data, source_labels, target_data, window_size=2048, step
            _size=1024):
21.            self.window_size = window_size
22.            self.step_size = step_size
23.            # 处理源域数据
24.            self.source_windows, self.source_labels = self._create_windows(
25.                source_data, source_labels, "源域"
26.            )
27.            # 处理目标域数据
28.            self.target_windows, _ = self._create_windows(
29.                target_data, None, "目标域"
30.            )
31.            # 创建域标签 (0=源域, 1=目标域)
32.            self.source_domain_labels = torch.zeros(len(self.source_windows))
33.            self.target_domain_labels = torch.ones(len(self.target_windows))
34.            # 合并所有数据
35.            self.all_windows = torch.cat([self.source_windows, self.target_windows], dim=0
            )
36.            self.domain_labels = torch.cat([self.source_domain_labels, self.target_domain_
            labels], dim=0)
37.            # 为源域数据创建类别标签, 目标域使用-1 表示无标签
38.            source_class_labels = torch.LongTensor(self.source_labels)
```

```

39.         target_class_labels = torch.full((len(self.target_windows),), -1, dtype=torch.
        long)
40.         self.class_labels = torch.cat([source_class_labels, target_class_labels], dim=
        0)
41.     def _create_windows(self, data, labels, domain_name):
42.         windows = []
43.         window_labels = []
44.         for i, sequence in enumerate(data):
45.             sequence = torch.FloatTensor(sequence)
46.             max_start = len(sequence) - self.window_size
47.             max_windows = 10
48.             step_count = 0
49.             for start in range(0, max_start + 1, self.step_size):
50.                 if step_count >= max_windows:
51.                     break
52.                 window = sequence[start:start + self.window_size]
53.                 windows.append(window)
54.                 step_count += 1
55.             windows = torch.stack(windows) if windows else torch.empty(0, self.window_size
        )
56.         return windows, window_labels
57.     def __len__(self):
58.         return len(self.all_windows)
59.     def __getitem__(self, idx):
60.         return {
61.             'data': self.all_windows[idx],
62.             'domain': self.domain_labels[idx],
63.             'class_label': self.class_labels[idx]
64.         }
65.     class DomainAdaptationViT(nn.Module):
66.         def __init__(self, seq_len=2048, patch_size=64, num_classes=4, embed_dim=256,
67.             depth=6, num_heads=8, dropout=0.1):
68.             super().__init__()
69.             self.seq_len = seq_len
70.             self.patch_size = patch_size
71.             self.num_patches = seq_len // patch_size
72.             self.embed_dim = embed_dim
73.             self.num_classes = num_classes
74.             self.patch_embed = nn.Linear(patch_size, embed_dim)
75.             # 位置编码
76.             self.pos_embed = nn.Parameter(torch.zeros(1, self.num_patches + 1, embed_dim))
77.             # CLS token
78.             self.cls_token = nn.Parameter(torch.zeros(1, 1, embed_dim))
79.             # Transformer blocks

```

```

80.         self.blocks = nn.ModuleList([
81.             nn.TransformerEncoderLayer(
82.                 d_model=embed_dim,
83.                 nhead=num_heads,
84.                 dim_feedforward=embed_dim * 4,
85.                 dropout=dropout,
86.                 activation='gelu',
87.                 batch_first=True
88.             ) for _ in range(depth)
89.         ])
90.         # 特征提取器
91.         self.feature_extractor = nn.Sequential(
92.             nn.LayerNorm(embed_dim), nn.Dropout(dropout)
93.         )
94.         # 分类器
95.         self.classifier = nn.Linear(embed_dim, num_classes)
96.         # 域判别器 (用于对抗训练, 可选)
97.         self._init_weights()
98.         def _init_weights(self):
99.             """初始化权重"""
100.            nn.init.trunc_normal_(self.pos_embed, std=0.02)
101.            nn.init.trunc_normal_(self.cls_token, std=0.02)
102.            for m in self.modules():
103.                if isinstance(m, nn.Linear):
104.                    nn.init.trunc_normal_(m.weight, std=0.02)
105.                    if m.bias is not None:
106.                        nn.init.constant_(m.bias, 0)
107.                elif isinstance(m, nn.LayerNorm):
108.                    nn.init.constant_(m.bias, 0)
109.                    nn.init.constant_(m.weight, 1.0)
110.
111.            def forward(self, x, return_features=False):
112.                """前向传播"""
113.                B = x.shape[0]
114.                # Patch embedding
115.                x = x.view(B, self.num_patches, self.patch_size)
116.                x = self.patch_embed(x)
117.                # 添加CLS token
118.                cls_tokens = self.cls_token.expand(B, -1, -1)
119.                x = torch.cat((cls_tokens, x), dim=1)
120.                # 位置编码
121.                x = x + self.pos_embed
122.                # Transformer blocks
123.                for block in self.blocks:

```

```

124.         x = block(x)
125.         # 提取CLS token 特征
126.         cls_features = x[:, 0]
127.         features = self.feature_extractor(cls_features)
128.         class_logits = self.classifier(features)
129.         # 域分类 (用于对抗训练)
130.         domain_logits = self.domain_classifier(features)
131.         return class_logits, domain_logits, features
132.     def mmd_loss(source_features, target_features, kernel_type='rbf', kernel_mul=2.0, kern
        el_num=5):
133.         batch_size = source_features.size(0)
134.         # 合并特征
135.         features = torch.cat([source_features, target_features], dim=0)
136.         # 计算核矩阵
137.         if kernel_type == 'rbf':
138.             # RBF 核
139.             n_samples = features.size(0)
140.             kernels = torch.zeros(n_samples, n_samples).to(features.device)
141.             for i in range(n_samples):
142.                 for j in range(n_samples):
143.                     diff = features[i] - features[j]
144.                     kernels[i, j] = torch.exp(-torch.norm(diff) ** 2 / (2 * 1.0 ** 2))
145.         else:
146.             # 线性核
147.             kernels = torch.mm(features, features.t())
148.             # 分离源域和目标域的核矩阵
149.             XX = kernels[:batch_size, :batch_size]
150.             YY = kernels[batch_size:, batch_size:]
151.             XY = kernels[:batch_size, batch_size:]
152.             # 计算MMD
153.             mmd = XX.mean() + YY.mean() - 2 * XY.mean()
154.             return mmd
155.     def coral_loss(source_features, target_features):
156.         """CORAL (Correlation Alignment) 损失"""
157.         # 计算特征的协方差矩阵
158.         def covariance(features):
159.             n = features.size(0)
160.             # 中心化
161.             features_centered = features - features.mean(dim=0)
162.             # 协方差
163.             cov = torch.mm(features_centered.t(), features_centered) / (n - 1)
164.             return cov
165.         # 源域和目标域的协方差矩阵
166.         source_cov = covariance(source_features)

```

```

167.     target_cov = covariance(target_features)
168.     # Frobenius 范数
169.     coral = torch.norm(source_cov - target_cov, p='fro') ** 2
170.     # 归一化
171.     d = source_features.size(1)
172.     coral = coral / (4 * d * d)
173.     return coral
174. def train_domain_adaptation(model, dataloader, optimizer, device, epoch, total_epochs)
    :
175.     """域适应训练"""
176.     model.train()
177.     total_loss = 0
178.     class_loss_total = 0
179.     mmd_loss_total = 0
180.     coral_loss_total = 0
181.     correct = 0
182.     total = 0
183.     for batch_idx, batch in enumerate(dataloader):
184.         data = batch['data'].to(device)
185.         domain_labels = batch['domain'].to(device)
186.         class_labels = batch['class_label'].to(device)
187.         optimizer.zero_grad()
188.         # 前向传播
189.         class_logits, domain_logits, features = model(data)
190.         # 分离源域和目标域
191.         source_mask = domain_labels == 0
192.         target_mask = domain_labels == 1
193.         source_features = features[source_mask]
194.         target_features = features[target_mask]
195.         source_class_labels = class_labels[source_mask]
196.         # 损失计算
197.         losses = {}
198.         # 1. 分类损失 (仅源域)
199.         if len(source_features) > 0:
200.             source_class_logits = class_logits[source_mask]
201.             class_loss = F.cross_entropy(source_class_logits, source_class_labels)
202.             losses['class'] = class_loss
203.             # 计算准确率
204.             _, predicted = torch.max(source_class_logits.data, 1)
205.             total += source_class_labels.size(0)
206.             correct += (predicted == source_class_labels).sum().item()
207.         # 2. MMD 损失
208.         if len(source_features) > 0 and len(target_features) > 0:
209.             mmd = mmd_loss(source_features, target_features)

```

```

210.         losses['mmd'] = mmd
211.         # 3. CORAL 损失
212.         if len(source_features) > 0 and len(target_features) > 0:
213.             coral = coral_loss(source_features, target_features)
214.             losses['coral'] = coral
215.         # 总损失
216.         total_batch_loss = 0
217.         if 'class' in losses:
218.             total_batch_loss += losses['class']
219.             class_loss_total += losses['class'].item()
220.         if 'mmd' in losses:
221.             total_batch_loss += 0.1 * losses['mmd'] # MMD 权重
222.             mmd_loss_total += losses['mmd'].item()
223.         if 'coral' in losses:
224.             total_batch_loss += 0.1 * losses['coral'] # CORAL 权重
225.             coral_loss_total += losses['coral'].item()
226.         # 反向传播
227.         if total_batch_loss > 0:
228.             total_batch_loss.backward()
229.             torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
230.             optimizer.step()
231.         total_loss += total_batch_loss.item()
232.         # 打印进度
233.         if batch_idx % 20 == 0:
234.             acc = 100. * correct / total if total > 0 else 0
235.             print(f'Epoch {epoch+1}/{total_epochs}, Batch {batch_idx}/{len(dataloader)}
    }, '
236.                 f'Loss: {total_batch_loss.item():.4f}, Acc: {acc:.2f}%')
237.         # 计算平均损失
238.         avg_loss = total_loss / len(dataloader)
239.         avg_class_loss = class_loss_total / len(dataloader)
240.         avg_mmd_loss = mmd_loss_total / len(dataloader)
241.         avg_coral_loss = coral_loss_total / len(dataloader)
242.         accuracy = 100. * correct / total if total > 0 else 0
243.         return avg_loss, avg_class_loss, avg_mmd_loss, avg_coral_loss, accuracy
244.     def evaluate_model(model, source_data, source_labels, target_data, device, label_encoder):
245.         """评估模型"""
246.         model.eval()
247.         results = {}
248.         with torch.no_grad():
249.             # 评估源域
250.             if len(source_data) > 0:
251.                 source_features = model(source_data.to(device), return_features=True)

```

```

252.         source_logits, _, _ = model(source_data.to(device))
253.         source_pred = torch.argmax(source_logits, dim=1).cpu().numpy()
254.         source_acc = accuracy_score(source_labels, source_pred)
255.         results['source_accuracy'] = source_acc
256.         # 预测目标域
257.         if len(target_data) > 0:
258.             target_logits, _, _ = model(target_data.to(device))
259.             target_pred = torch.argmax(target_logits, dim=1).cpu().numpy()
260.             results['target_predictions'] = target_pred
261.             # 预测分布
262.             unique, counts = np.unique(target_pred, return_counts=True)
263.             pred_dist = dict(zip(unique, counts))
264.             results['target_prediction_distribution'] = pred_dist
265.         return results
266.     def main():
267.         parser = argparse.ArgumentParser(description='ViT 域适应训练')
268.         parser.add_argument('--source_data', type=str, default='data4_resampled_32khz_balanced.csv')
269.         parser.add_argument('--target_data', type=str, default='target_process.csv')
270.         parser.add_argument('--batch_size', type=int, default=32)
271.         parser.add_argument('--epochs', type=int, default=50)
272.         parser.add_argument('--learning_rate', type=float, default=1e-4)
273.         parser.add_argument('--save_dir', type=str, default='domain_adaptation_results')
274.         args = parser.parse_args()
275.         device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
276.         os.makedirs(args.save_dir, exist_ok=True)
277.         # 加载源域数据
278.         source_df = pd.read_csv(args.source_data, header=0)
279.         source_features = source_df.iloc[:, :-1].values.astype(np.float32)
280.         source_labels = source_df.iloc[:, -1].values
281.         # 加载目标域数据
282.         print("加载目标域数据...")
283.         try:
284.             target_df = pd.read_csv(args.target_data, header=0)
285.             print("✓ 使用 header=0 读取目标域数据")
286.         except:
287.             target_df = pd.read_csv(args.target_data, header=None)
288.             print("✓ 使用 header=None 读取目标域数据")
289.         target_features = target_df.values.astype(np.float32)
290.         # 尝试对齐特征维度
291.         min_features = min(source_features.shape[1], target_features.shape[1])
292.         print(f" 调整为: {min_features} 特征")
293.
294.         source_features = source_features[:, :min_features]

```

```

295.         target_features = target_features[:, :min_features]
296.         # 标签编码
297.         label_encoder = LabelEncoder()
298.         source_labels_encoded = label_encoder.fit_transform(source_labels)
299.         num_classes = len(label_encoder.classes_)
300.         # 数据标准化
301.         scaler = StandardScaler()
302.         scaler.fit(source_features)
303.         source_features_scaled = scaler.transform(source_features)
304.         target_features_scaled = scaler.transform(target_features)
305.         # 创建域适应数据集
306.         dataset = DomainDataset(
307.             source_features_scaled, source_labels_encoded, target_features_scaled,
308.             window_size=2048, step_size=1024
309.         )
310.         # 创建数据加载器
311.         dataloader = DataLoader(
312.             dataset, batch_size=args.batch_size, shuffle=True, num_workers=0
313.         )
314.         # 创建域适应模型
315.         model = DomainAdaptationViT(
316.             seq_len=2048,
317.             patch_size=64,
318.             num_classes=num_classes,
319.             embed_dim=256,
320.             depth=6,
321.             num_heads=8
322.         ).to(device)
323.         total_params = sum(p.numel() for p in model.parameters())
324.         # 优化器和调度器
325.         optimizer = optim.AdamW(model.parameters(), lr=args.learning_rate, weight_decay=1e
-4)
326.         scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=args.epochs)
327.         # 训练记录
328.         train_history = {
329.             'total_loss': [],
330.             'class_loss': [],
331.             'mmd_loss': [],
332.             'coral_loss': [],
333.             'accuracy': []
334.         }
335.         best_accuracy = 0
336.         for epoch in range(args.epochs):
337.             print(f"\nEpoch {epoch+1}/{args.epochs}")

```

```

338.         print(f"学习率: {optimizer.param_groups[0]['lr']:.6f}")
339.         # 训练
340.         avg_loss, avg_class_loss, avg_mmd_loss, avg_coral_loss, accuracy = train_domain_adaptation(
341.             model, dataloader, optimizer, device, epoch, args.epochs
342.         )
343.         # 记录历史
344.         train_history['total_loss'].append(avg_loss)
345.         train_history['class_loss'].append(avg_class_loss)
346.         train_history['mmd_loss'].append(avg_mmd_loss)
347.         train_history['coral_loss'].append(avg_coral_loss)
348.         train_history['accuracy'].append(accuracy)
349.         # 保存最佳模型
350.         if accuracy > best_accuracy:
351.             best_accuracy = accuracy
352.             torch.save({
353.                 'epoch': epoch,
354.                 'model_state_dict': model.state_dict(),
355.                 'optimizer_state_dict': optimizer.state_dict(),
356.                 'accuracy': accuracy,
357.                 'label_encoder': label_encoder,
358.                 'scaler': scaler
359.             }, os.path.join(args.save_dir, 'best_domain_adaptation_model.pth'))
360.             scheduler.step()
361.         # 加载最佳模型
362.         checkpoint = torch.load(os.path.join(args.save_dir, 'best_domain_adaptation_model.pth'))
363.         model.load_state_dict(checkpoint['model_state_dict'])
364.         # 为评估准备数据
365.         source_windows = dataset.source_windows
366.         target_windows = dataset.target_windows
367.         source_labels = dataset.source_labels
368.         # 评估
369.         results = evaluate_model(model, source_windows, source_labels, target_windows, device, label_encoder)
370.         # 保存预测结果
371.         target_predictions = results['target_predictions']
372.         target_predictions_decoded = label_encoder.inverse_transform(target_predictions)
373.         # 保存到CSV
374.         prediction_df = pd.DataFrame({
375.             'sample_id': range(len(target_predictions)),
376.             'predicted_label': target_predictions_decoded,
377.             'predicted_class': target_predictions
378.         })

```

```

379.
380.     prediction_path = os.path.join(args.save_dir, 'target_predictions.csv')
381.     prediction_df.to_csv(prediction_path, index=False)
382.     # 保存训练历史
383.     history_path = os.path.join(args.save_dir, 'training_history.json')
384.     with open(history_path, 'w') as f:
385.         json.dump(train_history, f, indent=2)
386.     if __name__ == "__main__":
387.         main()

```

问题四:

```

1. import os
2. import json
3. import numpy as np
4. import pandas as pd
5. import matplotlib.pyplot as plt
6. import seaborn as sns
7. from sklearn.manifold import TSNE
8. from sklearn.preprocessing import StandardScaler, LabelEncoder
9. from sklearn.decomposition import PCA
10. import warnings
11.     warnings.filterwarnings('ignore')
12. # 设置matplotlib
13. import matplotlib

14. def load_and_preprocess_data(source_path, target_path):
15.     source_df = pd.read_csv(source_path, header=0)
16.     source_features = source_df.iloc[:, :-1].values.astype(np.float32)
17.     source_labels = source_df.iloc[:, -1].values
18.     target_df = pd.read_csv(target_path, header=0)
19.     target_features = target_df.values.astype(np.float32)
20.     # 特征维度对齐
21.     if source_features.shape[1] != target_features.shape[1]:
22.         min_features = min(source_features.shape[1], target_features.shape[1])
23.         source_features = source_features[:, :min_features]
24.         target_features = target_features[:, :min_features]
25.     # 标签编码
26.     label_encoder = LabelEncoder()
27.     source_labels_encoded = label_encoder.fit_transform(source_labels)
28.     # 创建目标域的虚拟标签 (用于可视化)
29.     target_labels_encoded = np.full(len(target_features), -1)
30.     # 数据标准化
31.     scaler = StandardScaler()
32.     source_features_scaled = scaler.fit_transform(source_features)
33.     target_features_scaled = scaler.transform(target_features)

```

```

34.     return (source_features_scaled, source_labels_encoded,
35.             target_features_scaled, target_labels_encoded,
36.             label_encoder)

37. def analyze_domain_separation(tsne_results, domain_labels, all_labels, label_encoder, save
    _dir):
38.     source_mask = domain_labels == 0
39.     target_mask = domain_labels == 1
40.
41.     source_points = tsne_results[source_mask]
42.     target_points = tsne_results[target_mask]
43.
44.     # 计算域间距离
45.     source_center = np.mean(source_points, axis=0)
46.     target_center = np.mean(target_points, axis=0)
47.     domain_distance = np.linalg.norm(target_center - source_center)
48.     # 计算域内方差
49.     source_variance = np.var(source_points, axis=0).mean()
50.     target_variance = np.var(target_points, axis=0).mean()
51.     # 计算分离度指标
52.     separation_ratio = domain_distance / (np.sqrt(source_variance) + np.sqrt(target_varian
        ce))
53.     unique_labels = np.unique(all_labels[source_mask])
54.     target_predictions = []
55.     for label in unique_labels:
56.         if label >= 0:
57.             class_mask = (all_labels == label) & source_mask
58.             class_points = tsne_results[class_mask]
59.             if len(class_points) > 0:
60.                 class_center = np.mean(class_points, axis=0)
61.                 class_variance = np.var(class_points, axis=0).mean()
62.                 # 计算该类别到目标域的平均距离
63.                 distances_to_target = []
64.                 for target_point in target_points:
65.                     dist = np.linalg.norm(target_point - class_center)
66.                     distances_to_target.append(dist)
67.
68.                 avg_dist_to_target = np.mean(distances_to_target) if distances_to_target e
                    lse float('inf')
69.                 class_name = str(label_encoder.inverse_transform([label])[0])
70.                 # 预测目标域样本的类别
71.                 for i, target_point in enumerate(target_points):
72.                     min_distance = float('inf')
73.                     predicted_class = 'Unknown'

```

```

74.     for label in unique_labels:
75.     for label in unique_labels:
76.         if label >= 0:
77.             class_mask = (all_labels == label) & source_mask
78.             class_points = tsne_results[class_mask]
79.             if len(class_points) > 0:
80.                 class_center = np.mean(class_points, axis=0)
81.                 distance = np.linalg.norm(target_point - class_center)
82.
83.                 if distance < min_distance:
84.                     min_distance = distance
85.                     predicted_class = str(label_encoder.inverse_transform([label])[0])
86.                 target_predictions.append(predicted_class)
87.     # 保存分析结果
88.     analysis_results = {
89.         'domain_distance': float(domain_distance),
90.         'source_center': [float(x) for x in source_center.tolist()],
91.         'target_center': [float(x) for x in target_center.tolist()],
92.         'source_variance': float(source_variance),
93.         'target_variance': float(target_variance),
94.         'separation_ratio': float(separation_ratio),
95.         'target_predictions': {
96.             f'target_sample_{i+1}': {
97.                 'predicted_class': str(pred),
98.                 'tsne_x': float(target_points[i, 0]),
99.                 'tsne_y': float(target_points[i, 1])
100.            } for i, pred in enumerate(target_predictions)
101.        }
102.    }
103.    with open(os.path.join(save_dir, 'domain_analysis.json'), 'w') as f:
104.        json.dump(analysis_results, f, indent=2)
105.    return analysis_results
106. def main():
107.     # 加载和预处理数据
108.     (source_features, source_labels,
109.      target_features, target_labels,
110.      label_encoder) = load_and_preprocess_data(args.source_data, args.target_data)
111.     # 创建t-SNE 可视化
112.     tsne_results, domain_labels, all_labels = create_tsne_visualization(
113.         source_features, source_labels, target_features, target_labels,
114.         label_encoder, args.save_dir
115.     )
116.     # 分析域分离情况

```

```
117.         analysis_results = analyze_domain_separation(tsne_results, domain_labels, all_labels, label_encoder, args.save_dir)
118.         # 创建详细分析图
119.         create_detailed_analysis_plots(tsne_results, domain_labels, all_labels, label_encoder, args.save_dir)
120. if __name__ == "__main__":
121.     main()
```